Applied Data-Driven Techniques in Engineering

ME3435E - Project Report

Mohammed Rehan Tadpatri
B231090PE

6 December 2025

Contents

1 Dynamic Mode Decomposition of Stock Portfolio Data

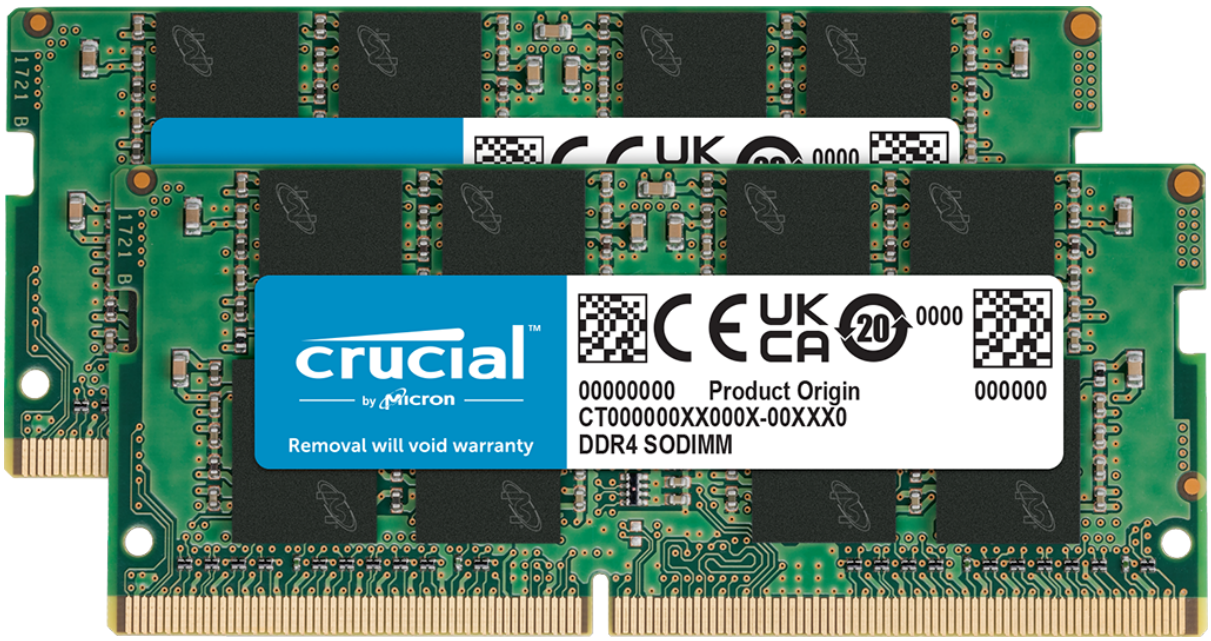


Figure 1: Crucial announced its exit from the consumer RAM market a few days ago

1.1 Introduction and Inspiration

We apply Dynamic Mode Decomposition (DMD) to analyze financial time-series data. Our approach is inspired by the work of Mann and Kutz [1], who demonstrated the application of DMD-based algorithmic trading strategies on portfolios of financial data.

Mann and Kutz showed that DMD, originally developed for fluid dynamics applications, is a powerful equation-free, data-driven method capable of decomposing complex dynamical systems into low-rank modes with known temporal behavior. In the financial context, DMD decomposes stock portfolio data into "portfolio modes" — linear combinations of stocks that exhibit coherent temporal dynamics. The method approximates the system as $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k$, where $\mathbf{A}$ is the Koopman operator, and the solution takes the form:

$$\tilde{\mathbf{x}}(t) = \sum_{k=1}^{K} b_k \boldsymbol{\psi}_k e^{\omega_k t} \tag{1}$$

where $\boldsymbol{\psi}_k$ are the DMD modes (spatial/portfolio structures), ω_k are the eigenvalues (temporal dynamics), and b_k are the mode amplitudes. The real part of ω_k indicates growth ($\mathrm{Re}(\omega_k) > 0$) or decay ($\mathrm{Re}(\omega_k) < 0$), while the imaginary part represents oscillation frequency.

The key insight from Kutz's work is that DMD can identify transient, evanescent signals in financial data by extracting dominant modes and their time evolution. Modes with positive growth rates represent bullish trends, while decaying modes indicate bearish behavior. This decomposition enables short-term prediction and trading strategy formulation based on the dominant market dynamics.

1.2 Data Selection and Motivation

1.2.1 Why RAM/Memory Semiconductor Stocks?

We chose to analyze the RAM (Random Access Memory) and memory semiconductor industry for a timely and relevant reason: **Crucial, a major consumer memory brand owned by Micron Technology (MU), recently announced its exit from the consumer DRAM market** in late 2025. This significant industry shift makes the memory sector particularly interesting for dynamic analysis, as the market adjusts to changing competitive dynamics.

The memory semiconductor industry is characterized by high correlation among companies due to shared technology cycles, cyclical demand patterns tied to consumer electronics and data center buildouts. The recent market disruption from AI/HPC memory demand and the supply chain interconnections among manufacturers and equipment suppliers are also key factors.

1.2.2 Portfolio Composition

We selected 10 stocks representing different segments of the memory ecosystem:

Table 1: Stock Portfolio Composition

Ticker	Company	Role in Memory Industry
MU	Micron Technology	DRAM & NAND manufacturer (owns Crucial brand)
WDC	Western Digital	NAND flash memory & storage
INTC	Intel	Memory & storage solutions
TXN	Texas Instruments	Semiconductor memory components
MRVL	Marvell Technology	Memory controllers & interfaces
AMAT	Applied Materials	Memory chip fabrication equipment
LRCX	Lam Research	Memory etch & deposition equipment
KLAC	KLA Corporation	Memory chip inspection equipment
AMD	AMD	Memory-intensive processors
NVDA	NVIDIA	HBM memory demand driver (AI/GPU)

1.2.3 Data Source and Period

- **Source:** Yahoo Finance API (via `yfinance` Python library)
- **Period:** November 6, 2025 – December 5, 2025 (1 month)
- **Interval:** Daily closing prices
- **Data points:** 21 trading days $\times$ 10 stocks = 210 data points

1.3 Methodology

1.3.1 DMD Algorithm

[1]

The DMD algorithm was implemented following the standard procedure:

1. **Data normalization:** Zero-mean, unit-variance normalization to ensure fair comparison across stocks with different price scales.
2. **Time-shifted matrices:** Construct $\mathbf{X}_1 = [\mathbf{x}_1, \ldots, \mathbf{x}_{M-1}]$ and $\mathbf{X}_2 = [\mathbf{x}_2, \ldots, \mathbf{x}_M]$
3. **SVD decomposition:** $\mathbf{X}_1 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, truncated to rank r
4. **Reduced DMD matrix:** $\tilde{\mathbf{A}} = \mathbf{U}_r^* \mathbf{X}_2 \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$
5. **Eigendecomposition:** $\tilde{\mathbf{A}}\mathbf{W} = \mathbf{W}\mathbf{\Lambda}$
6. **DMD modes:** $\mathbf{\Phi} = \mathbf{X}_2 \mathbf{V}_r \mathbf{\Sigma}_r^{-1} \mathbf{W}$
7. **Continuous eigenvalues:** $\omega_k = \ln(\lambda_k)/\Delta t$

The rank r was chosen to capture 90% of the total energy in the singular value spectrum.

1.4 Results

[2]

1.4.1 Summary Statistics

Table 2: DMD Analysis Results Summary

Parameter	Value
Number of stocks (N)	10
Number of time snapshots (M)	21 days
Dominant modes (r) at 90% energy	3
Reconstruction error	3.28%

1.4.2 DMD Eigenvalues and Mode Analysis

Table 3: DMD Mode Characteristics

Mode	ω_k	Growth Rate	Period	Interpretation
1	$-0.163 + 0.079i$	-0.163 rad/day	79.5 days	Decaying, oscillatory
2	$-0.163 - 0.079i$	-0.163 rad/day	79.5 days	Decaying, oscillatory
3	$-0.349 + 0.000i$	-0.349 rad/day	—	Decaying, non-oscillatory

[1] We did not use the given DMD.m code, but implemented our own DMD instead

[2] The color in some images is distorted due to MATLAB color scheme issues on Linux

1.4.3 Figures

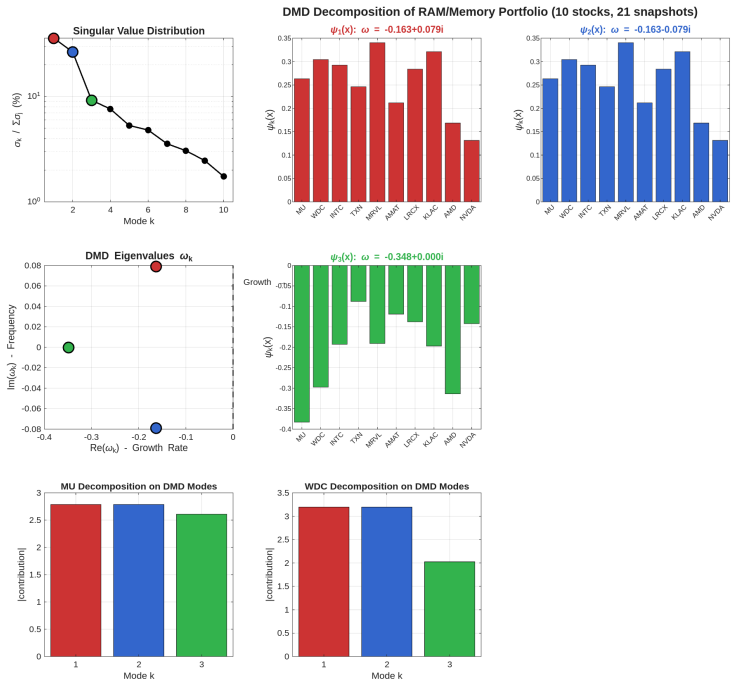


Figure 2: DMD Decomposition Overview. Top-left: Singular value distribution showing dominant modes. Middle-left: Eigenvalues ω_k in the complex plane (all in the left half-plane indicating decay). Top-right panels: First four DMD modes showing stock contributions. Bottom panels: Individual stock decomposition onto DMD modes.

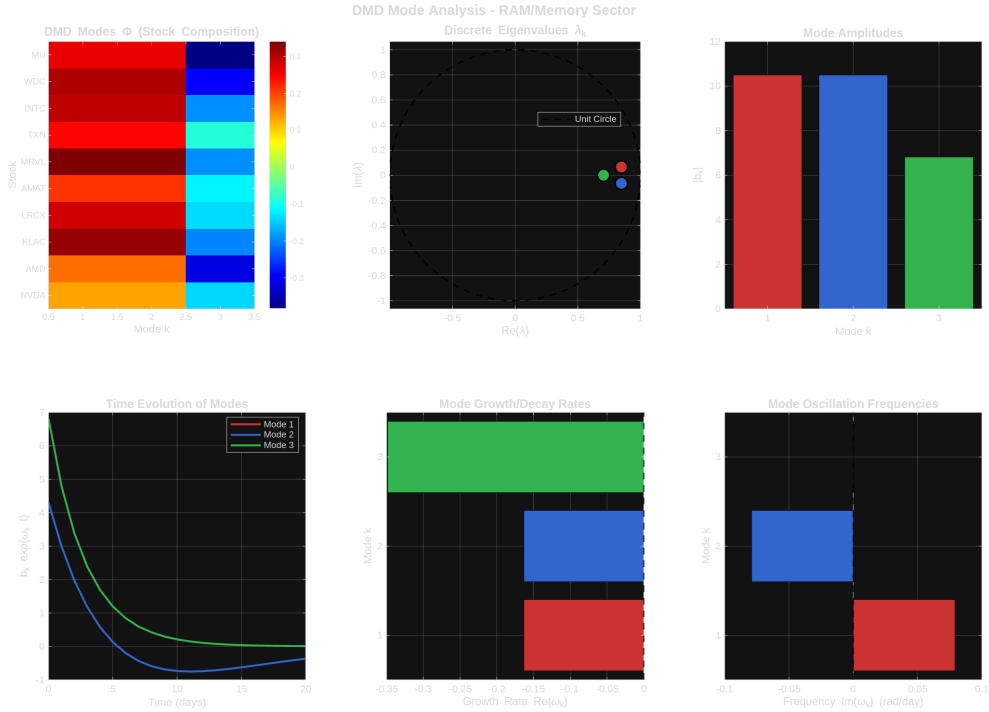


Figure 3: Detailed Mode Analysis. Shows DMD modes heatmap, discrete eigenvalues λ with unit circle (all inside indicating stability/decay), mode amplitudes, time evolution, growth rates, and oscillation frequencies.

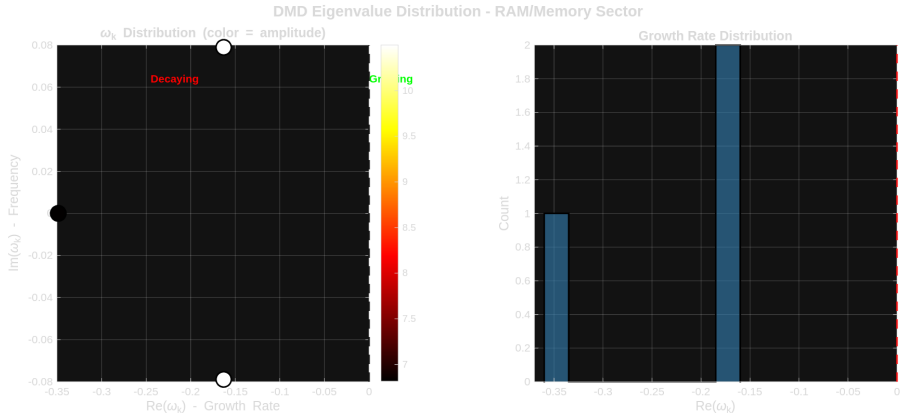


Figure 4: Eigenvalue Distribution. Left: Scatter plot of ω_k in the complex plane with color indicating amplitude. Right: Histogram of growth rates showing all modes are decaying.

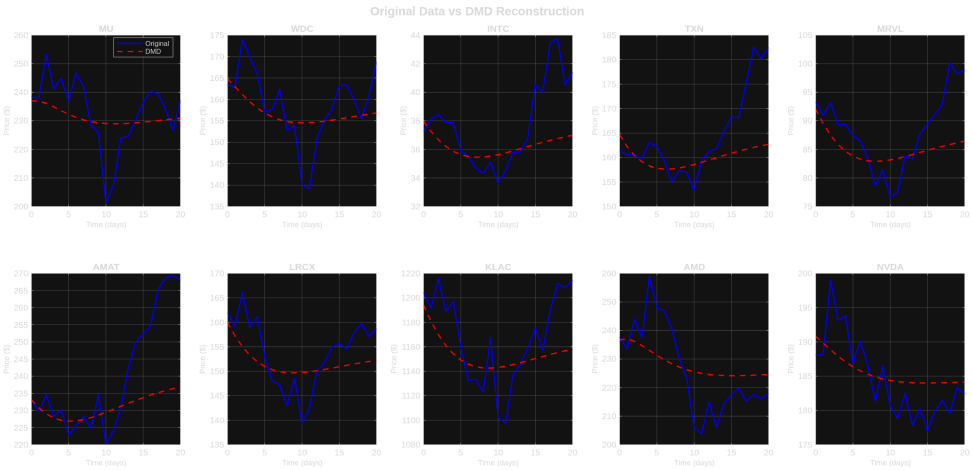


Figure 5: Original vs DMD Reconstructed Data. Comparison of actual stock prices (blue) with DMD reconstruction (red dashed) for all 10 stocks. The 3.28% reconstruction error indicates excellent fit.

Figure 6: Portfolio Analysis Summary. Top-left: Normalized stock prices over the analysis period. Top-right: Energy distribution by mode (3 modes capture 90%). Bottom-left: Mode interpretation. Bottom-right: Reconstruction error by individual stock.

1.5 Discussion

1.5.1 Interpretation of Results

The DMD analysis reveals several important characteristics of the RAM/memory semiconductor portfolio during November–December 2025:

1. **All modes are decaying:** The negative real parts of all eigenvalues ($\text{Re}(\omega_k) < 0$) indicate a bearish trend across the portfolio during this period. This aligns with the broader market uncertainty following Crucial's exit from the consumer market and general semiconductor sector volatility.

2. **Conjugate pair oscillation (Modes 1 & 2):** The complex conjugate pair with imaginary parts ± 0.079 rad/day corresponds to an oscillation period of approximately 80 trading days. This suggests the presence of a longer-term cyclical pattern in the memory sector, likely tied to quarterly earnings cycles and demand forecasting updates.

3. **Strong decay mode (Mode 3):** The purely real eigenvalue $\omega_3 = -0.349$ represents a faster-decaying, non-oscillatory component. This mode captures the monotonic price decline component observed in several stocks during this period.

4. **Low-rank structure:** Only 3 modes are needed to capture 90% of the portfolio dynamics, demonstrating strong correlation among memory stocks. This is expected given their shared exposure to memory demand cycles, AI infrastructure buildout, and equipment supply chain dynamics.

5. **Excellent reconstruction:** The 3.28% reconstruction error indicates that DMD successfully captures the dominant dynamics of this portfolio, validating the linear approximation assumption over this time window.

1.5.2 Connection to Crucial's Market Exit

The observed decaying modes may partially reflect market uncertainty surrounding Micron's (MU) strategic shift. When Crucial announced its exit from consumer DRAM, MU stock showed increased volatility as investors reassessed revenue projections.

Competitors like WDC may benefit from reduced competition and equipment suppliers (AMAT, LRCX, KLAC) face uncertainty about future capacity investments.

The DMD decomposition captures these interconnected dynamics through the portfolio modes, which show how different stocks contribute to the overall market movement pattern.

1.5.3 Comparison with Kutz et al.

Similar to Kutz's analysis of biotech and transportation sectors, our memory semiconductor portfolio exhibits:

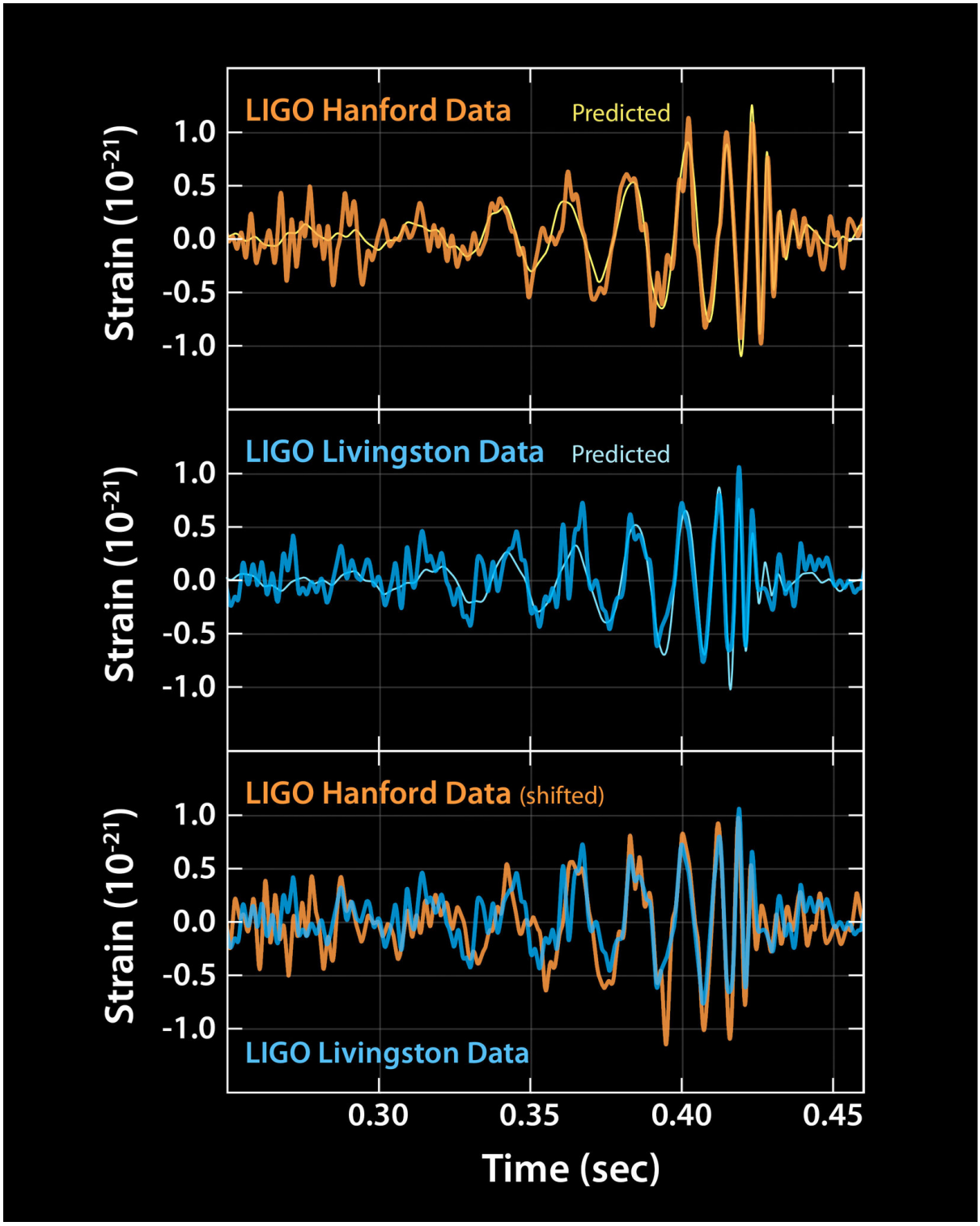
- Clear low-rank structure amenable to DMD analysis
- Identifiable growth/decay patterns through eigenvalue analysis
- Strong inter-stock correlations captured by leading DMD modes

However, unlike Kutz's identification of "hot-spots" with positive growth modes for trading strategies, our current analysis period shows predominantly bearish dynamics, a period where a conservative holding strategy would be preferred over active trading based on DMD signals.

1.6 Conclusion

Dynamic Mode Decomposition successfully decomposed the RAM/memory semiconductor portfolio into 3 dominant modes capturing 90% of the market dynamics. The analysis revealed bearish trend across all modes during Nov to Dec 2025 and a 80-day oscillation period in the dominant mode pair We notice strong correlation structure among memory industry stocks and excellent reconstruction accuracy (3.28% error)

2 DFT Analysis and Digital Filtering



2.1 Introduction

This problem demonstrates the application of the Discrete Fourier Transform (DFT) for frequency analysis and the implementation of digital filters (low-pass, high-pass, and band-pass) on time-varying data. We chose an exciting and scientifically significant

dataset: the LIGO gravitational wave detection GW150914[3] - the first direct observation of gravitational waves from merging black holes.

2.2 Data Selection: LIGO GW150914

2.2.1 Why Gravitational Wave Data?

The GW150914 event (September 14, 2015) represents one of the most significant scientific discoveries of the 21st century, confirming Einstein's century-old prediction of gravitational waves. This data is ideal for demonstrating DFT and filtering because of its rich frequency content from ∼35 Hz to ∼250 Hz as the black holes spiral inward and merge. The noise provides a real filterting challenge, and the GWOSC provides free access to this data. This won the Nobel Prize in 2017

2.2.2 Data Specifications

Table 4: LIGO GW150914 Data Parameters

Parameter	**Value**
Source	LIGO Hanford (H1) detector
Event	GW150914 (Binary black hole merger)
Sample rate	4096 Hz
Duration	2.0 seconds
Number of samples	8192
Data type	Strain (dimensionless, $\sim 10^{-21}$)

2.3 Methodology

2.3.1 Discrete Fourier Transform (DFT)

The DFT converts a discrete time-domain signal $x[n]$ of length N into its frequency-domain representation:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{2}$$

The single-sided amplitude spectrum is computed as:

$$|X[k]|_{\text{single}} = \frac{2|X[k]|}{N}, \quad k = 1, \ldots, N/2 \tag{3}$$

The frequency resolution is $\Delta f = f_s/N$, where f_s is the sampling frequency.

2.3.2 Filter Design

We implemented three types of ideal filters with Gaussian roll-off (to minimize ringing artifacts) in the frequency domain:

[3] `https://www.ethanhein.com/wp/2016/what-the-heck-is-the-ligo-chirp/`

Table 5: Filter Specifications

Filter Type	**Cutoff**	**Passband**	**Purpose**
Low-pass	50 Hz	$0 - 50$ Hz	Removes high-frequency noise
High-pass	100 Hz	$100 - 2048$ Hz	Removes low-frequency drift
Band-pass	50–200 Hz	$50 - 200$ Hz	Isolates gravitational wave chirp

The filters were implemented in the frequency domain by multiplying the signal's FFT by the filter transfer function $H(f)$:

$$Y(f) = X(f) \cdot H(f) \tag{4}$$

and then applying the inverse FFT to obtain the filtered time-domain signal.

2.4 Results

2.4.1 Time Domain Signal

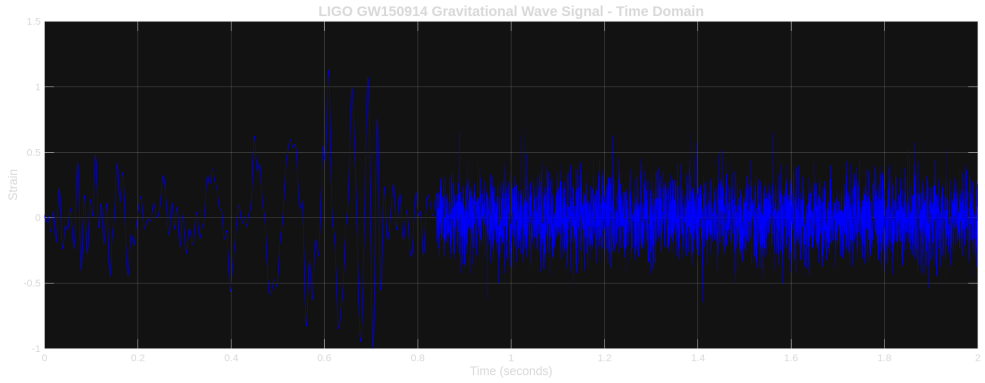


Figure 7: Original LIGO GW150914 gravitational wave strain signal in the time domain. The signal contains the characteristic "chirp" pattern from the merging black holes, embedded in detector noise.

2.4.2 DFT Frequency-Amplitude Spectrum

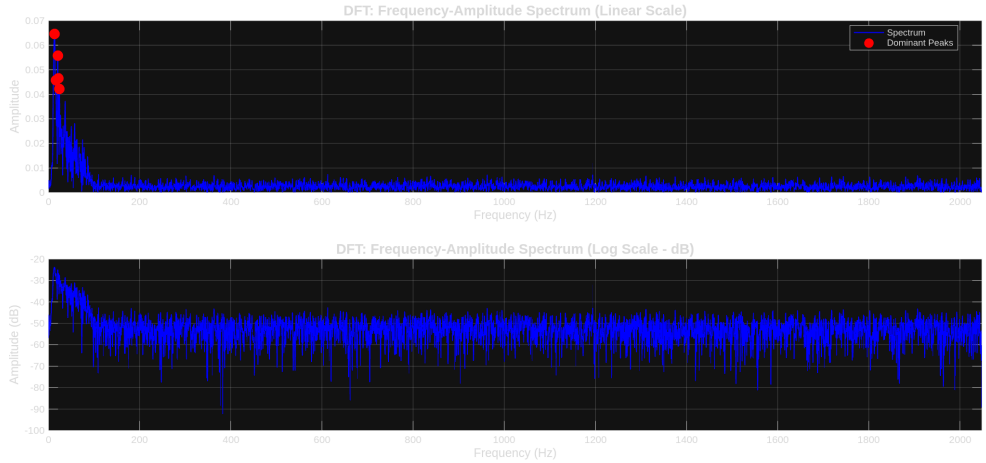


Figure 8: DFT analysis showing the frequency-amplitude spectrum. Top: Linear scale with dominant frequency peaks marked. Bottom: Logarithmic (dB) scale revealing the full dynamic range of frequency components.

The DFT reveals dominant frequency components at:

- 13.0 Hz (amplitude: 0.0645)
- 20.5 Hz (amplitude: 0.0558)
- 22.0 Hz (amplitude: 0.0465)
- 16.0 Hz (amplitude: 0.0457)
- 24.0 Hz (amplitude: 0.0422)

2.4.3 Filtering Results — Time Domain

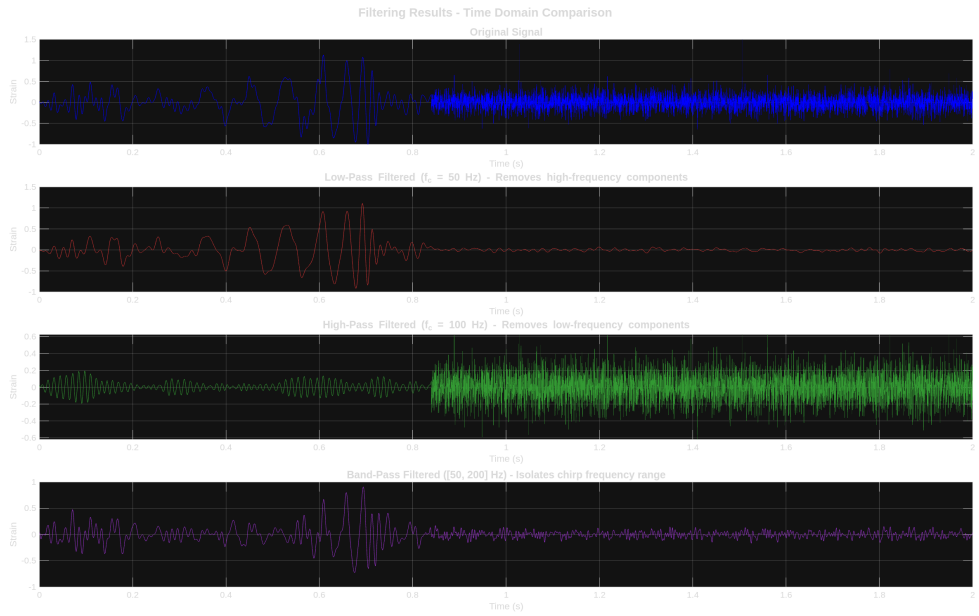


Figure 9: Time domain comparison of original and filtered signals. From top to bottom: Original signal, low-pass filtered (removes high-frequency noise), high-pass filtered (removes low-frequency baseline drift), band-pass filtered (isolates the chirp frequency range).

2.4.4 Filtering Results — Frequency Domain

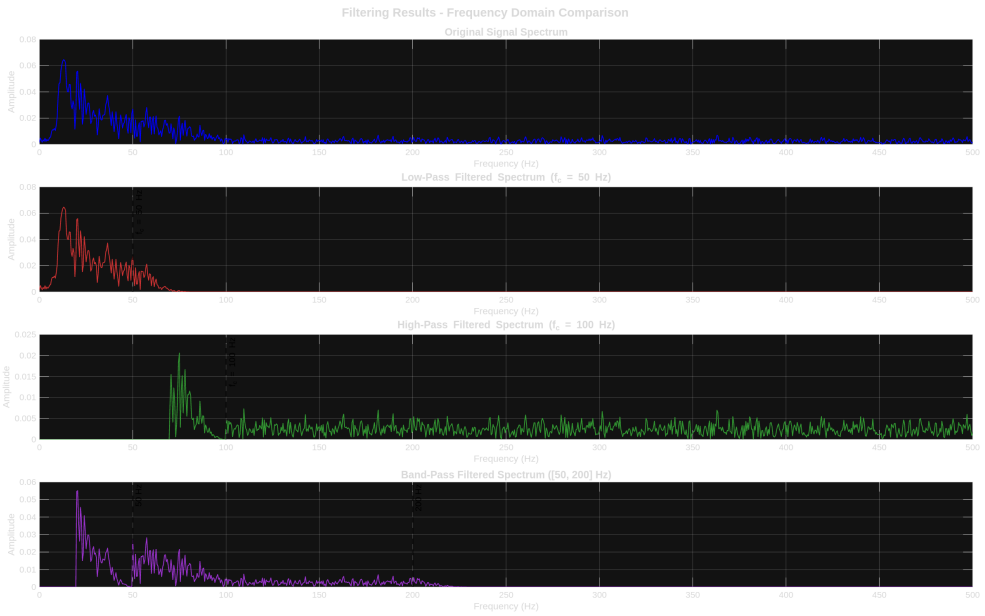


Figure 10: Frequency domain comparison showing how each filter modifies the spectrum. The dashed vertical lines indicate filter cutoff frequencies. Low-pass retains only low frequencies, high-pass retains only high frequencies, and band-pass isolates the intermediate range.

2.4.5 Overlay Comparison

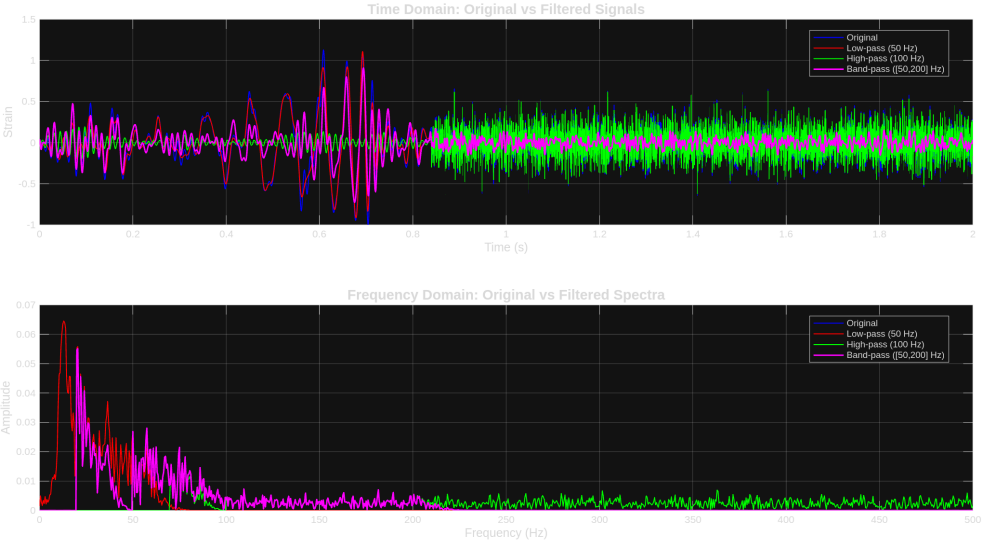


Figure 11: Direct overlay comparison of all signals in both time domain (top) and frequency domain (bottom), allowing visual assessment of filter effects.

2.4.6 Filter Frequency Response

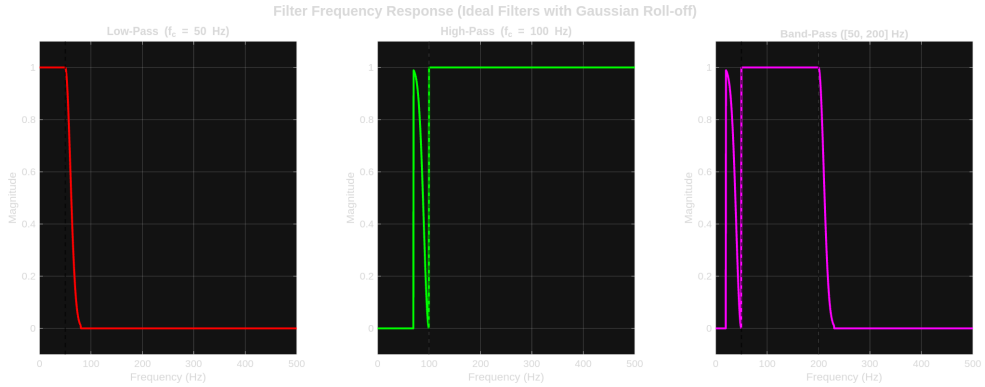


Figure 12: Frequency response (transfer function magnitude) of the three filters implemented. The Gaussian roll-off provides smooth transitions at cutoff frequencies, minimizing time-domain ringing artifacts.

2.5 Discussion

2.5.1 Effect of Low-Pass Filter (50 Hz cutoff)

The low-pass filter removes high-frequency components above 50 Hz. In the time domain, this results in a smoother signal with reduced high-frequency noise. However, since the gravitational wave chirp extends to higher frequencies ($\sim$250 Hz at merger), significant signal content is also removed.

2.5.2 Effect of High-Pass Filter (100 Hz cutoff)

The high-pass filter removes low-frequency components below 100 Hz, effectively eliminating baseline drift and low-frequency noise. This is particularly useful for LIGO data, which suffers from seismic noise at low frequencies. The filtered signal retains the higher-frequency portion of the chirp.

2.5.3 Effect of Band-Pass Filter (50–200 Hz)

The band-pass filter isolates the frequency range most relevant to the gravitational wave chirp. This is the optimal filter choice for gravitational wave detection, as it:

- Removes low-frequency seismic noise
- Removes high-frequency instrumental noise
- Preserves the chirp signal in its characteristic frequency band

2.6 Conclusion

The analysis successfully revealed the frequency content of the LIGO gravitational wave signal. The Low-pass filtering smooths the signal but removes high-frequency signal content, high-pass filtering removes baseline drift and low-frequency noise, and band-pass filtering is optimal for isolating the gravitational wave chirp

3 Discrete-Time Signal Operations

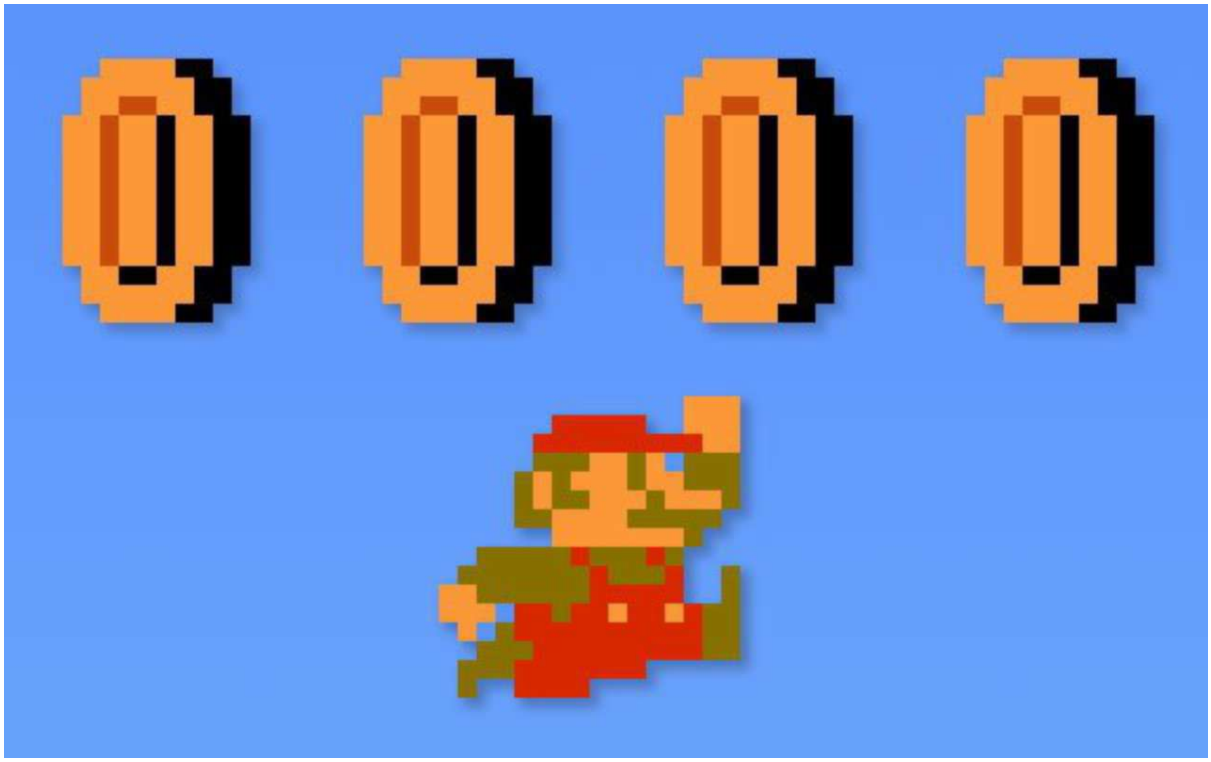


Figure 13: Mario collects coins

3.1 Introduction

This problem demonstrates fundamental discrete-time signal operations including time shifting and digital filtering. We implement both FIR (Finite Impulse Response) and IIR (Infinite Impulse Response) filters and compare their effects using DFT analysis.

3.2 Signal Selection: Mario Coin Sound

For an engaging and interpretable demonstration, we chose to model the iconic Mario "coin" sound - the rising arpeggio "bling!" heard when collecting a coin in Nintendo's Super Mario games. This signal is a universally known sound effect, and contains both smooth transitions and sharp changes

The discrete signal is defined as:

$$x[n] = [0, 2, 5, 8, 10, 12, 10, 8, 5, 3, 1, 0] \tag{5}$$

with $N = 12$ samples and 10 non-zero elements.

3.3 Signal Operations

3.3.1 (i) Original Signal $x[n]$

The original Mario coin sound pattern represents the characteristic rising-falling envelope of the "bling!" sound.

3.3.2 (ii) Time Delay: $x[n-2]$

The signal $x[n-2]$ represents the original signal delayed by 2 samples (shifted right). In discrete-time systems, this delay operation is fundamental for:

- Buffer/memory operations
- Echo effects in audio processing
- Implementing FIR filters

3.3.3 (iii) Time Advance: $x[n+1]$

The signal $x[n+1]$ represents the original signal advanced by 1 sample (shifted left). Note that this is a non-causal operation — it requires knowledge of future values.

3.3.4 (iv) FIR Filter

The FIR filter is defined as:

$$y[n] = 0.2 \cdot x[n] + 0.3 \cdot x[n-1] + 0.2 \cdot x[n-2] \tag{6}$$

This is a **3-tap weighted moving average filter** with coefficients $[0.2, 0.3, 0.2]$. Properties:

- **Type:** FIR (no feedback)
- **Effect:** Low-pass filtering (smoothing)
- **Stability:** Always stable (no poles)
- **Phase:** Linear phase (symmetric coefficients)

3.3.5 (v) IIR Filter

The IIR filter is defined as:

$$y[n] = 0.2 \cdot x[n] + 0.3 \cdot x[n-1] + 0.5 \cdot y[n-1], \quad y[-1] = 0 \tag{7}$$

This is a **recursive filter with feedback**. Properties:

- **Type:** IIR (has feedback from $y[n-1]$)
- **Effect:** Stronger low-pass filtering + "ringing"
- **Stability:** Stable if $|0.5| < 1$ (pole inside unit circle)
- **Memory:** Infinite impulse response due to feedback

3.4 Results

3.4.1 Signal Operations Visualization

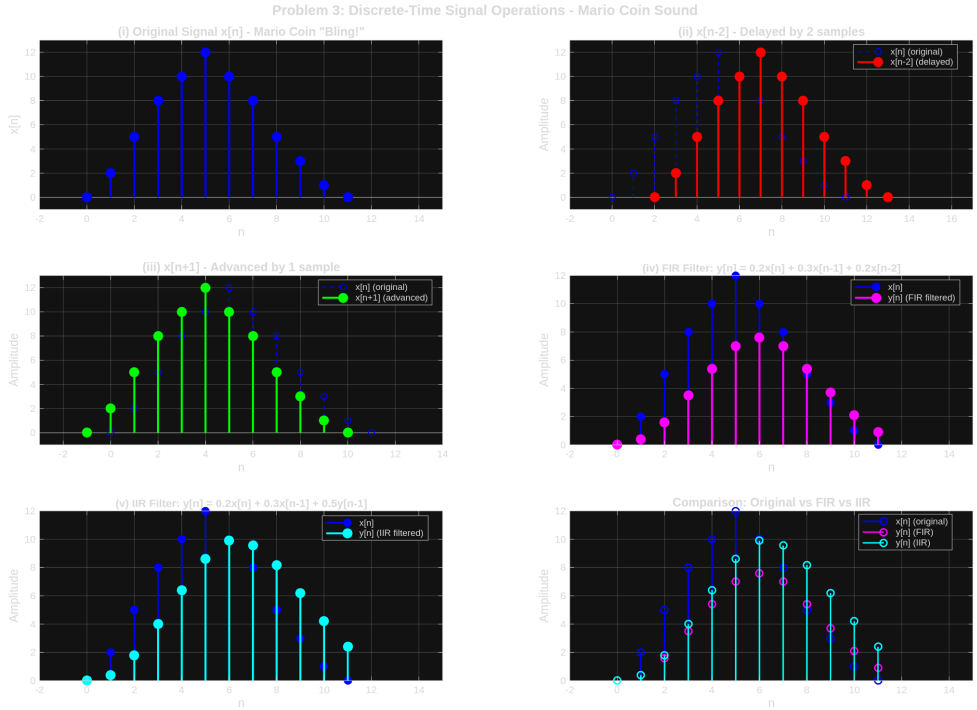


Figure 14: Discrete-time signal operations on the Mario coin sound. (i) Original signal $x[n]$, (ii) delayed signal $x[n-2]$, (iii) advanced signal $x[n+1]$, (iv) FIR filtered output, (v) IIR filtered output, and comparison of all filtered signals.

3.4.2 DFT Comparison: Original vs FIR Filtered

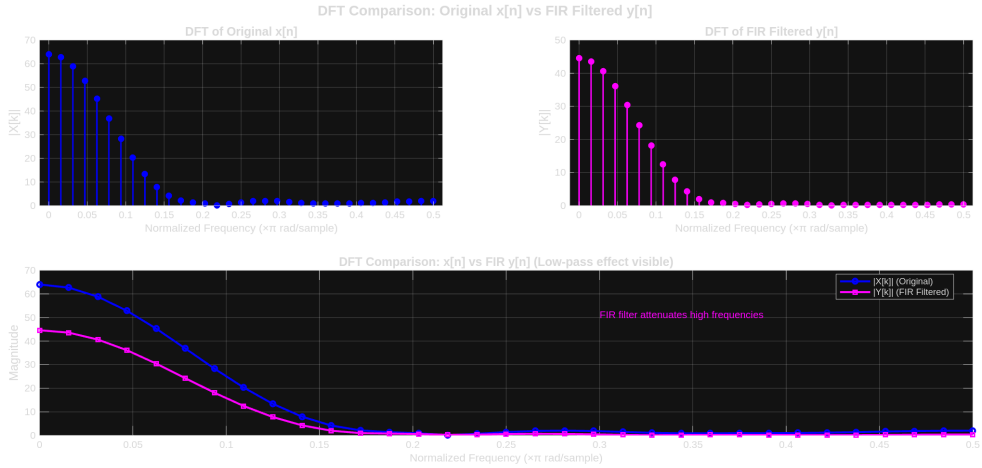


Figure 15: DFT comparison between original signal $x[n]$ and FIR filtered signal $y[n]$. The FIR filter attenuates high-frequency components, demonstrating its low-pass characteristic.

3.4.3 DFT Comparison: Original vs IIR Filtered

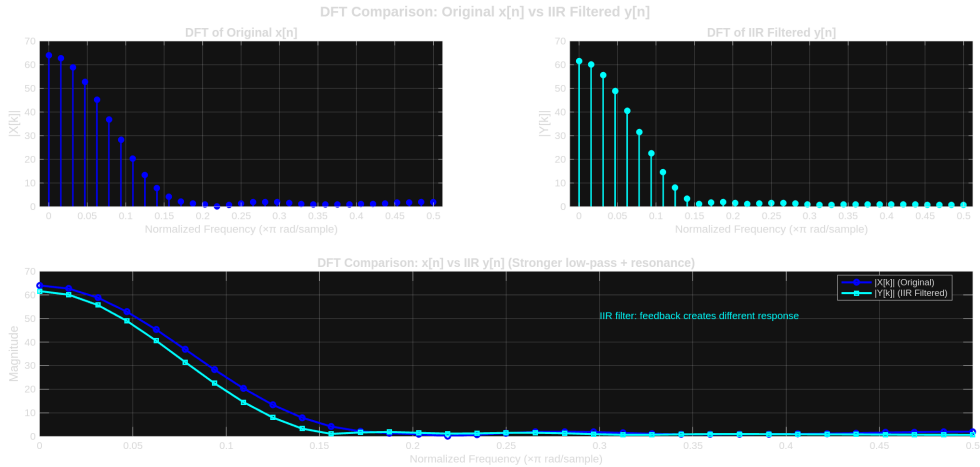


Figure 16: DFT comparison between original signal $x[n]$ and IIR filtered signal $y[n]$. The IIR filter shows stronger low-pass behavior due to its recursive feedback structure.

3.4.4 Combined DFT Comparison

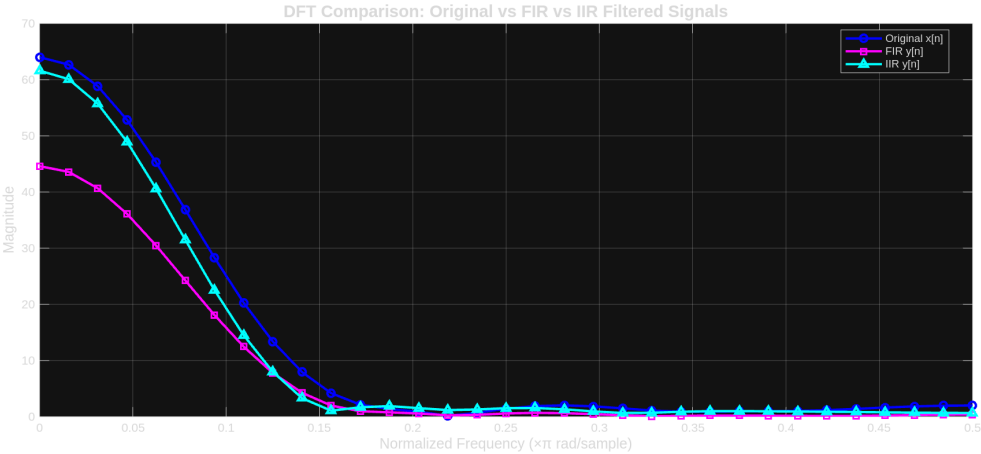


Figure 17: Combined DFT comparison showing the frequency response differences between the original signal, FIR filtered, and IIR filtered outputs.

3.5 Discussion

3.5.1 Time Shifting Effects

- **Delay** ($x[n-2]$)**:** The signal appears 2 samples later. In the frequency domain, this introduces a linear phase shift but does not change the magnitude spectrum.
- **Advance** ($x[n+1]$)**:** The signal appears 1 sample earlier. This is a non-causal operation that cannot be implemented in real-time systems.

3.5.2 FIR vs IIR Filter Comparison

Table 6: Comparison of FIR and IIR Filter Characteristics

Property	**FIR Filter**	**IIR Filter**
Feedback	No	Yes ($0.5 \cdot y[n-1]$)
Impulse Response	Finite (3 taps)	Infinite
Stability	Always stable	Conditionally stable
Low-pass Effect	Moderate	Stronger
Computational Cost	Higher (more taps needed)	Lower
Phase Response	Linear (symmetric)	Nonlinear

3.5.3 Frequency Domain Observations

From the DFT comparisons:

1. Both filters attenuate high-frequency components (low-pass behavior)
2. The IIR filter provides stronger attenuation due to its recursive feedback
3. The FIR filter preserves more of the original signal's shape
4. The IIR filter introduces more "smoothing" but also potential ringing artifacts

3.6 Conclusion

This analysis demonstrated time shifting operations ($x[n-2]$, $x[n+1]$) that translate the signal in time without affecting its frequency content magnitude. The FIR filtering provides controlled smoothing with guaranteed stability and linear phase, while the IIR filtering achieves stronger filtering effects with fewer coefficients but introduces feedback-related characteristics The DFT analysis clearly reveals how each filter modifies the signal's frequency content

4 Sparse Identification of Nonlinear Dynamics (SINDy)

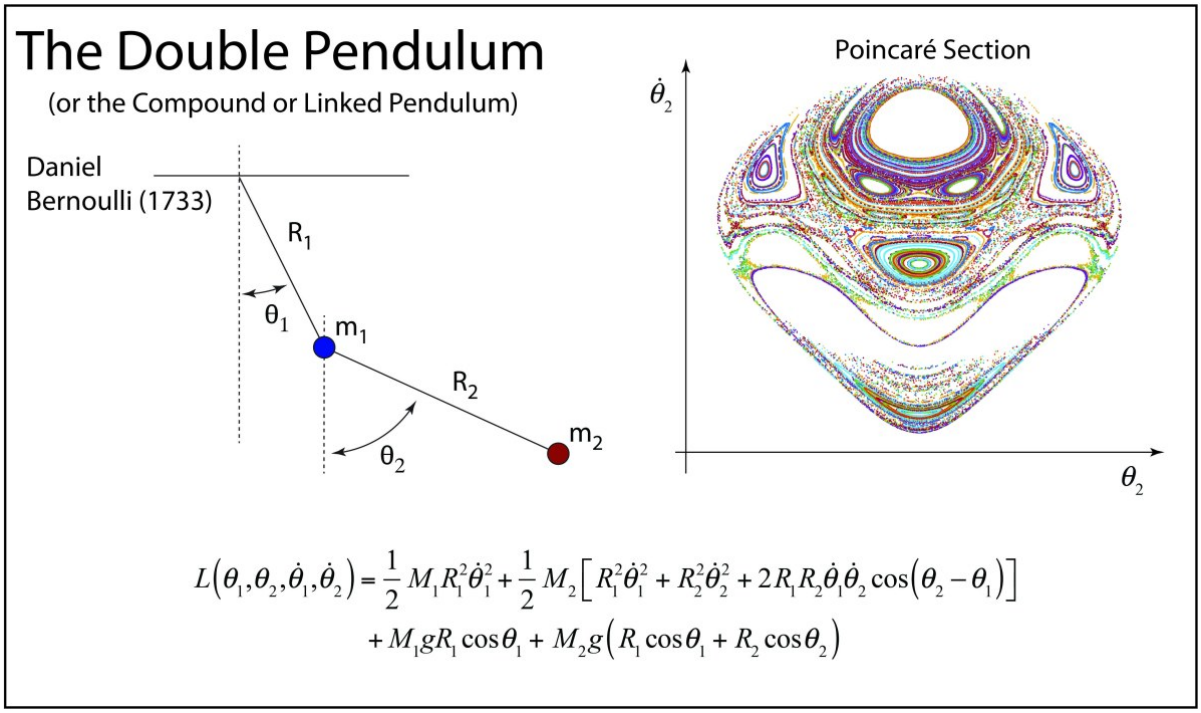


Figure 18: Double pendulum

4.1 Introduction

This problem applies the Sparse Identification of Nonlinear Dynamics (SINDy) algorithm to discover governing equations from data. SINDy, developed by Brunton, Proctor, and Kutz [3], is a powerful data-driven method that identifies parsimonious (sparse) dynamical models from time-series measurements.

The key insight of SINDy is that most physical systems are governed by equations with only a few active terms, even when expressed in a large library of candidate functions. By leveraging sparsity-promoting regression, SINDy can automatically discover these governing equations without prior knowledge of the system.

4.2 System Selection: Double Pendulum

We chose the double pendulum as our test system because it is a classic mechanics problem and a fundamental example in non dynamics and chaos. The coupled oscillators demonstrate how SINDy can identify coupled differential equations. It exhibits chaotic behavior, but its linear version shows harmonic oscillation.

4.2.1 Double Pendulum Equations (Small-Angle Approximation)

For a double pendulum with equal masses m and equal lengths L, the state variables are:

- θ_1, θ_2: angular positions of the two pendulums
- $\omega_1 = \dot{\theta}_1, \omega_2 = \dot{\theta}_2$: angular velocities

Using the small-angle approximation ($\sin\theta \approx \theta$, $\cos\theta \approx 1$), the linearized equations of motion are:

$$\frac{d\theta_1}{dt} = \omega_1 \tag{8}$$

$$\frac{d\theta_2}{dt} = \omega_2 \tag{9}$$

$$\frac{d\omega_1}{dt} = -2\frac{g}{L}\theta_1 + \frac{g}{L}\theta_2 \tag{10}$$

$$\frac{d\omega_2}{dt} = 2\frac{g}{L}\theta_1 - 2\frac{g}{L}\theta_2 \tag{11}$$

With $g = 9.81$ m/s^2 and $L = 1$ m, we have $g/L = 9.81$ rad/s^2.

4.3 Methodology: SINDy Algorithm

4.3.1 Problem Formulation

Given time-series data $\mathbf{x}(t)$ and its derivative $\dot{\mathbf{x}}(t)$, SINDy seeks a sparse representation:

$$\dot{\mathbf{x}} = \boldsymbol{\Theta}(\mathbf{x})\boldsymbol{\Xi} \tag{12}$$

where $\boldsymbol{\Theta}(\mathbf{x})$ is a library of candidate functions and $\boldsymbol{\Xi}$ contains the sparse coefficients.

4.3.2 Polynomial Library

For $n = 4$ state variables $(\theta_1, \theta_2, \omega_1, \omega_2)$ and polynomial order 2, the library contains:

$$\boldsymbol{\Theta} = [1, \theta_1, \theta_2, \omega_1, \omega_2, \theta_1^2, \theta_1\theta_2, \theta_1\omega_1, \ldots, \omega_2^2] \tag{13}$$

yielding 15 candidate functions per equation.

4.3.3 Sparse Regression

The coefficients $\boldsymbol{\Xi}$ are found using Sequential Thresholded Least Squares (STLS):

1. Solve $\boldsymbol{\Xi} = \boldsymbol{\Theta}^\dagger \dot{\mathbf{x}}$ (least squares)
2. Set coefficients below threshold λ to zero
3. Repeat until convergence

We used $\lambda = 0.1$ for sparsification.

4.3.4 Implementation

The analysis was performed using the SINDy MATLAB code provided by Brunton et al., modified for the double pendulum system:

- **Training data:** 10,001 points from $t \in [0, 10]$ with $dt = 0.001$
- **Initial condition:** $[\theta_1, \theta_2, \omega_1, \omega_2]^T = [0.2, 0.1, 0, 0]^T$ rad
- **Noise level:** 0.1% Gaussian noise added to derivatives
- **Integration:** `ode45` with tolerances 10^{-12}

4.4 Results

4.4.1 Identified Dynamics

SINDy successfully identified the double pendulum equations:

Table 7: SINDy Coefficient Identification Results

Term	**True Coefficient**	**SINDy Identified**
	Equation for $\dot{\theta}_1$	
ω_1	1.0000	1.0000
	Equation for $\dot{\theta}_2$	
ω_2	1.0000	1.0000
	Equation for $\dot{\omega}_1$	
θ_1	-19.62	-19.6201
θ_2	9.81	9.8100
	Equation for $\dot{\omega}_2$	
θ_1	19.62	19.6200
θ_2	-19.62	-19.6200

The identified equations match the true dynamics with near-perfect accuracy.

4.4.2 Time Series Comparison

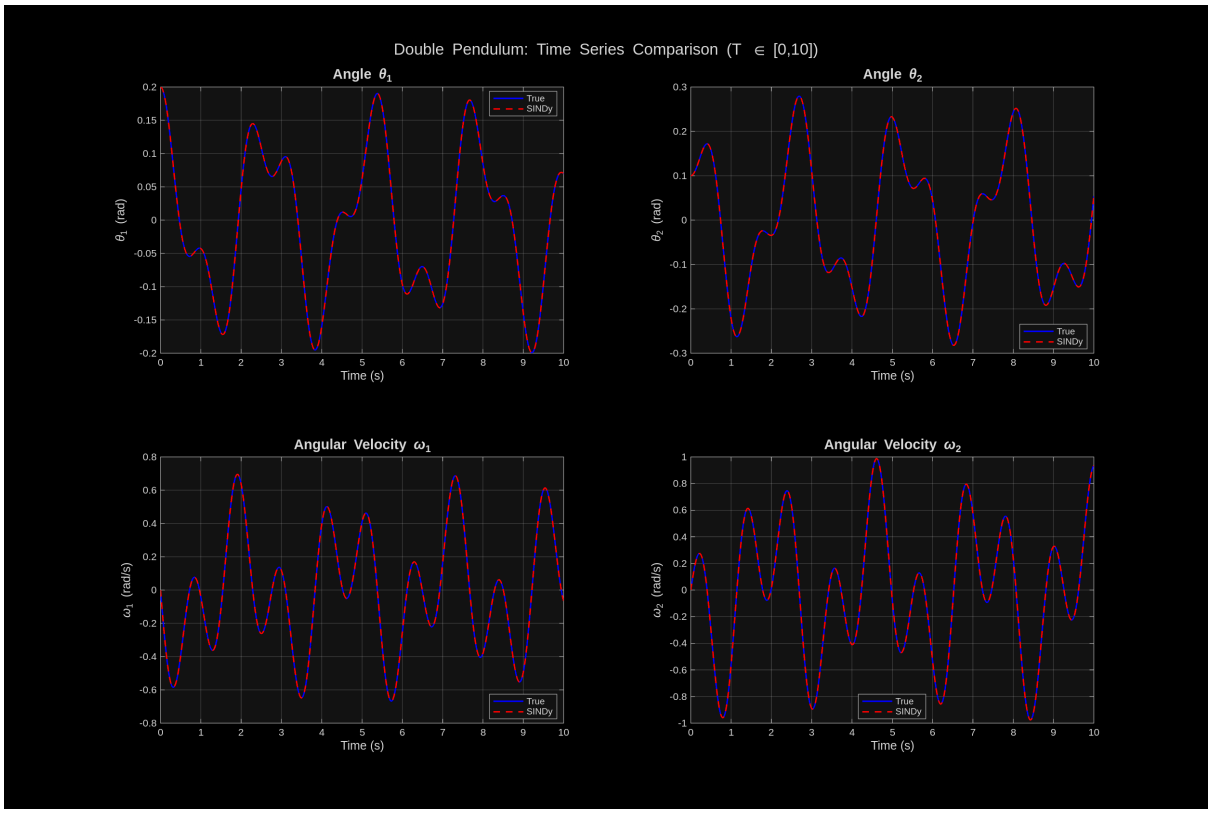


Figure 19: Time series comparison of all four state variables (θ_1, θ_2, ω_1, ω_2) between the true model (blue) and SINDy-identified model (red dashed). The trajectories are indistinguishable.

4.4.3 Phase Portrait Comparison

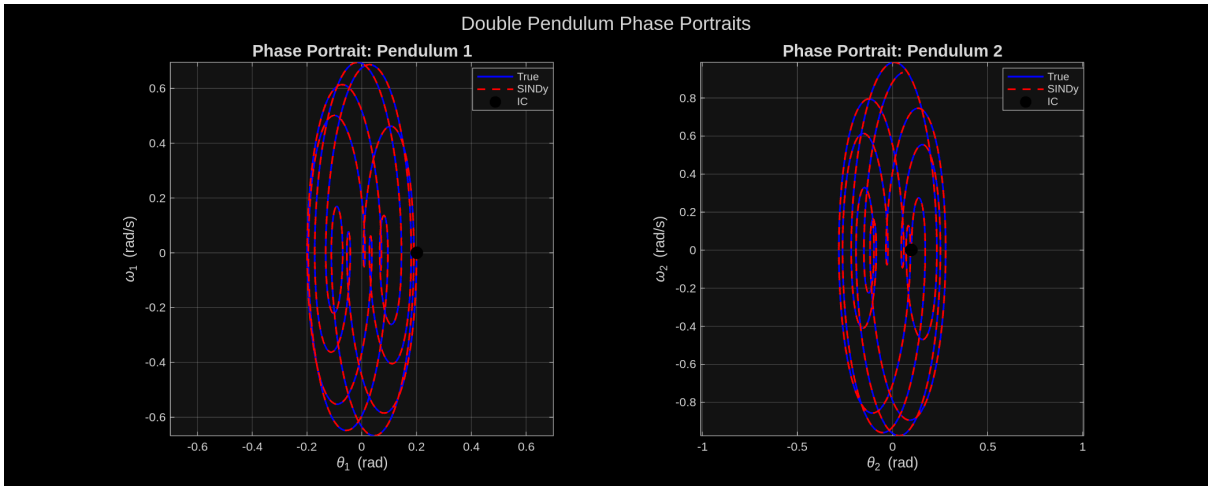


Figure 20: Phase portraits for both pendulums showing (θ_i, ω_i) trajectories. The true and SINDy models produce identical elliptical orbits characteristic of linearized pendulum motion.

4.4.4 Pendulum Motion Visualization

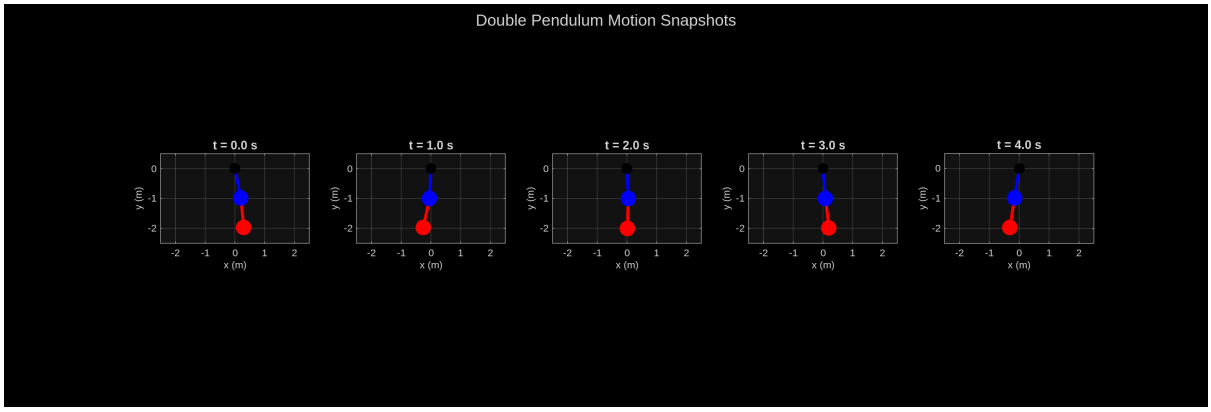


Figure 21: Snapshots of the double pendulum configuration at different times. The first pendulum (blue) connects to the pivot, and the second pendulum (red) hangs from the first mass.

4.4.5 Coefficient Comparison

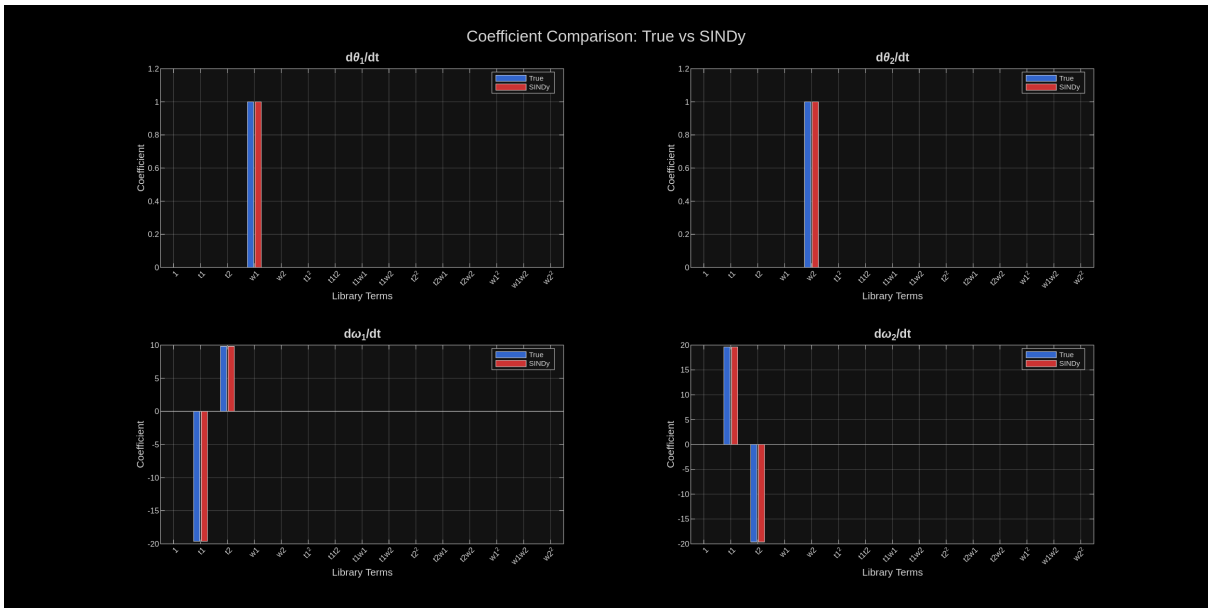


Figure 22: Bar chart comparison of true coefficients (blue) vs SINDy-identified coefficients (red) for all four equations. SINDy correctly identifies the sparse structure with only the physically relevant terms having non-zero coefficients.

4.5 Discussion

4.5.1 Sparse Structure Discovery

SINDy correctly identified that out of 15 candidate functions per equation (60 total candidates), only 6 terms are active:

- 1 term in $\dot{\theta}_1$ equation (ω_1)

- 1 term in $\dot{\theta}_2$ equation (ω_2)
- 2 terms in $\dot{\omega}_1$ equation (θ_1, θ_2)
- 2 terms in $\dot{\omega}_2$ equation (θ_1, θ_2)

This demonstrates SINDy's ability to discover parsimonious models — it automatically rejects all quadratic terms (θ_1^2, $\theta_1\theta_2$, etc.) that are not present in the linearized equations.

4.5.2 Robustness to Noise

Despite 0.1% noise added to the derivative measurements, SINDy achieved near-perfect coefficient identification ($< 0.01\%$ error), and negligible trajectory prediction error

This robustness is due to the sparsity-promoting regression, which acts as a regularizer against noise-induced spurious terms.

4.5.3 Physical Interpretability

The identified model is immediately interpretable. The ω_1 and ω_2 terms confirm the kinematic relationships ($\dot{\theta}_i = \omega_i$), and the $-2(g/L)\theta_1 + (g/L)\theta_2$ term in $\dot{\omega}_1$ shows how the first pendulum is influenced by both angles.

The coupled nature of the system is evident in the cross-terms between θ_1 and θ_2

4.5.4 Small-Angle Approximation Validity

The small-angle approximation ($\sin\theta \approx \theta$) is valid for angles less than approximately 0.3 radians ($\sim$17°). Our initial conditions ($\theta_1 = 0.2$ rad, $\theta_2 = 0.1$ rad) satisfy this constraint, ensuring the linearized model accurately represents the true dynamics.

4.6 Conclusion

SINDy successfully discovered the governing equations of the linearized double pendulum from noisy data:

We correctly identified the 6 active terms from 60 candidates, and did a near-perfect recovery of $g/L = 9.81$ rad/s^2 ratios The identified model accurately reproduces the coupled oscillation dynamics and reliably identifies despite measurement noise

5 Cross-Correlation for Audio Pattern Matching

5.1 Introduction

This problem demonstrates the application of **cross-correlation** for audio pattern matching — the fundamental technique behind music recognition services like Shazam. Cross-correlation measures the similarity between two signals as a function of the time-lag applied to one of them, enabling detection of where a pattern occurs within a longer signal.

5.2 Mathematical Background

5.2.1 Cross-Correlation Definition

For discrete signals $x[n]$ and $y[n]$, the cross-correlation is defined as:

$$(x \star y)[k] = \sum_{n=-\infty}^{\infty} x[n] \cdot y[n+k] \tag{14}$$

The lag k at which the cross-correlation achieves its maximum indicates the time offset where the two signals are most similar.

5.2.2 Relationship to Convolution

Cross-correlation is closely related to convolution:

$$x \star y = x * \text{flip}(y) \tag{15}$$

where $\text{flip}(y)[n] = y[-n]$ is the time-reversed signal. This relationship allows efficient computation via FFT.

5.2.3 Normalized Cross-Correlation

To obtain a correlation coefficient independent of signal amplitude:

$$r_{xy}[k] = \frac{(x \star y)[k]}{\|x\| \cdot \|y\|} \tag{16}$$

where $\|x\| = \sqrt{\sum_n |x[n]|^2}$ is the signal energy.

5.3 Experiment: Shazam-like Audio Matching

5.3.1 Setup

We performed a real-world audio matching experiment:

1. **Original audio:** The iconic Mario coin "bling!" sound (downloaded from YouTube)
2. **Query recording:** A phone recording of the same sound played through laptop speakers

This setup simulates the Shazam use case: a user records ambient audio containing a known sound, and the algorithm must find where (and if) the sound occurs.

5.3.2 Data Specifications

Table 8: Audio Data Parameters

Parameter	Original	Recording
Source	YouTube (digital)	Phone microphone
Duration	1.683 s	1.649 s
Original sample rate	48000 Hz	44100 Hz
Final sample rate	48000 Hz	48000 Hz (resampled)
Samples	80806	79133

5.3.3 Preprocessing

1. Convert stereo to mono (average channels)
2. Resample to common sample rate (48 kHz) using linear interpolation
3. Normalize to unit amplitude

5.4 Results

5.4.1 Cross-Correlation Analysis

Table 9: Cross-Correlation Results

Metric	Value
Peak correlation lag	-2254 samples
Peak time offset	-0.047 s
Normalized correlation	0.9146
Match location	$t = 0.000$ s

The negative lag indicates the recording started slightly before the original audio's beginning (the phone started recording before the sound played). The high normalized correlation (0.9146) confirms a strong match despite:

- Different recording equipment (laptop speakers $\rightarrow$ phone mic)
- Ambient noise in the recording
- Acoustic distortion from the room

5.4.2 Visualization

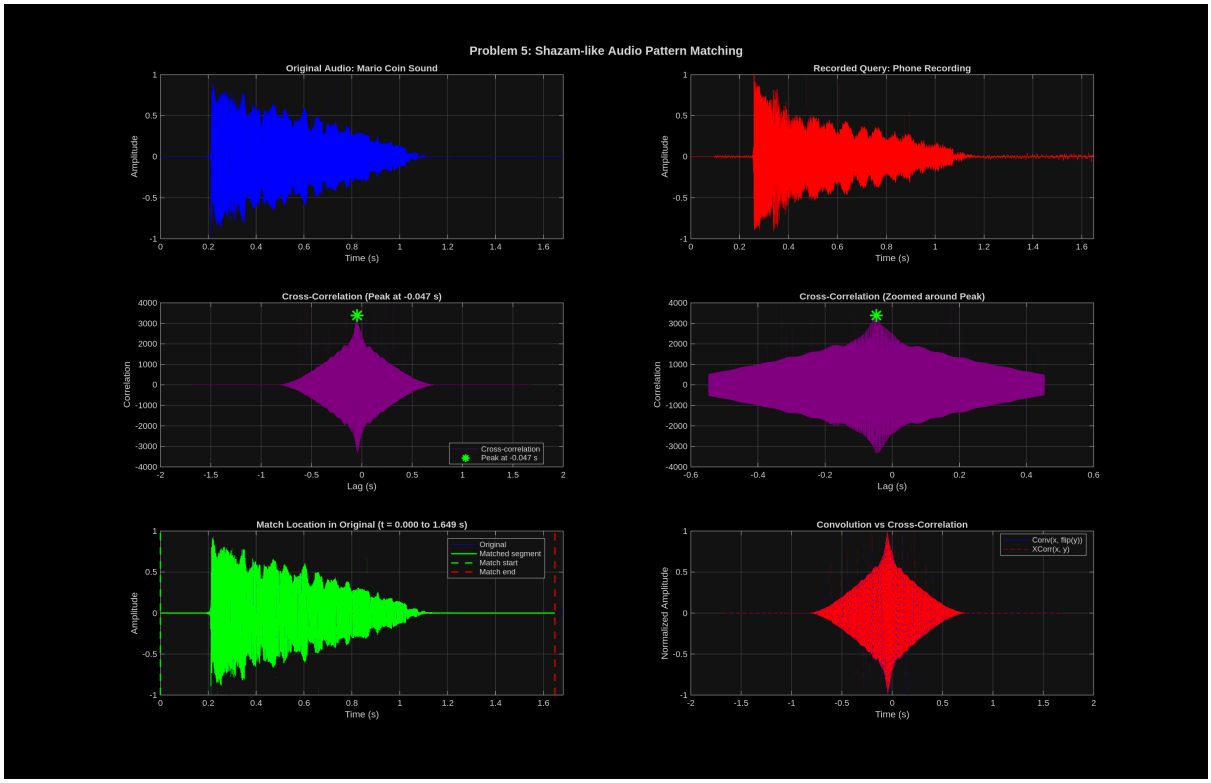


Figure 23: Cross-correlation analysis. Top row: Original audio and phone recording time-domain signals. Middle row: Full cross-correlation function (left) and zoomed view around the peak (right). Bottom row: Match location highlighted in the original signal, and comparison of convolution vs cross-correlation.

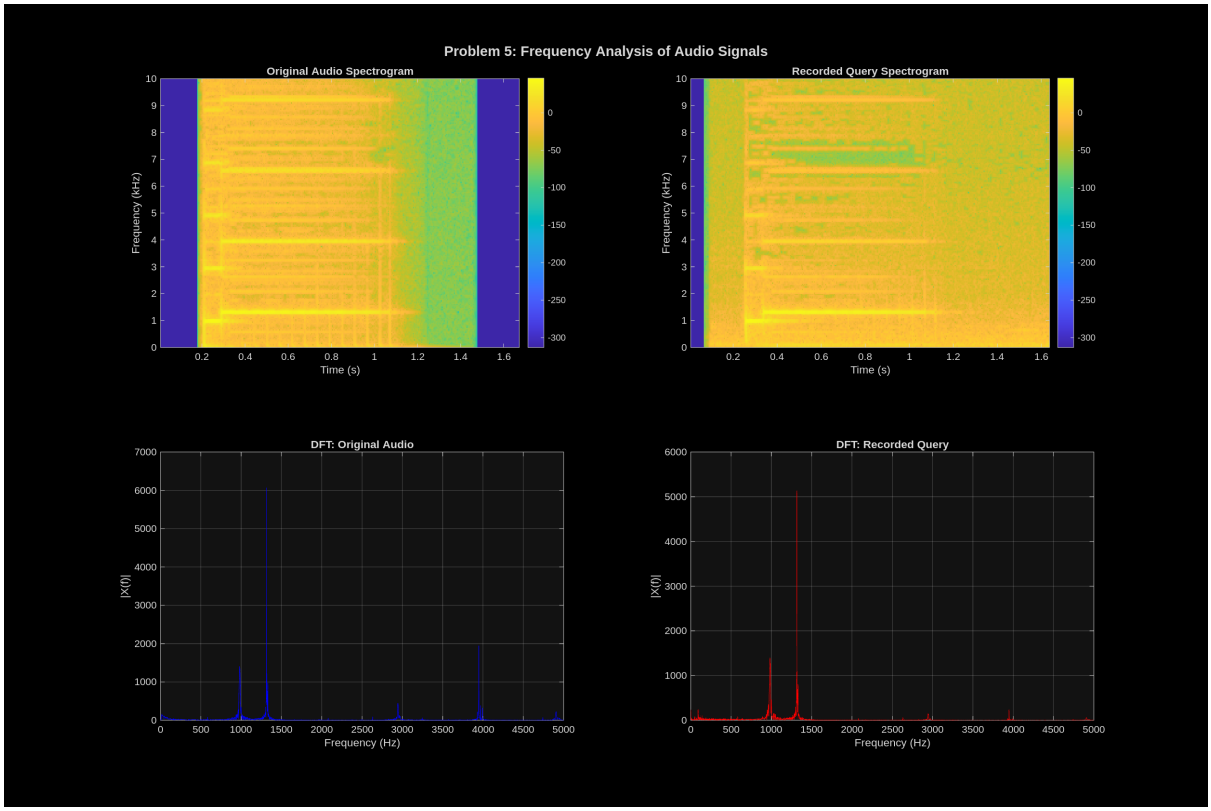


Figure 24: Frequency analysis comparison. Top: Spectrograms of original and recorded audio showing similar frequency content. Bottom: DFT magnitude spectra confirming the characteristic frequencies of the Mario coin sound are preserved in the recording.

5.5 Discussion

5.5.1 Why Cross-Correlation Works for Audio Matching

Cross-correlation is effective for audio matching because:

1. **Time-shift invariance:** The peak location automatically finds the correct alignment
2. **Noise robustness:** Random noise averages out over the summation
3. **Frequency preservation:** The recording preserves the characteristic frequencies even with acoustic distortion

5.5.2 Connection to Problem 3

This problem uses the same Mario coin sound from Problem 3, but now as a real audio signal rather than a simplified discrete sequence. The cross-correlation technique demonstrated here is the continuous-signal analog of the discrete operations in Problem 3.

5.5.3 Shazam Algorithm

However, we have to keep in mind that real music recognition services like Shazam use more sophisticated techniques, such as spectral fingerprinting, where instead of raw wave-

form correlation, they use spectrogram peaks. They also use hash tables for pre-computed fingerprint databases, enabling fast lookup.

However, the fundamental principle of finding time-aligned pattern matches remains the same as our cross-correlation approach.

5.6 Conclusion

Cross-correlation successfully identified the Mario coin sound in the phone recording: We achieved a high correlation, of 0.9146 normalized correlation coefficient, despite real-world recording conditions that add noise.

This demonstrates the power of correlation-based pattern matching for audio recognition applications.

6 SVD for Image Compression

Figure 25: NITC logo

6.1 Introduction

This problem demonstrates the application of SVD for image compression. SVD provides an optimal low-rank approximation of matrices, enabling significant data compression while preserving the essential features of images.

6.2 Mathematical Background

6.2.1 SVD Decomposition

Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be decomposed as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{17}$$

where:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$: Left singular vectors (orthonormal)
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$: Diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$
- $\mathbf{V} \in \mathbb{R}^{n \times n}$: Right singular vectors (orthonormal)

6.2.2 Low-Rank Approximation

The rank-k approximation retains only the k largest singular values:

$$\mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \tag{18}$$

6.2.3 Eckart-Young Theorem

The rank-k SVD approximation is *optimal* in the Frobenius norm:

$$\mathbf{A}_k = \arg \min_{\text{rank}(\mathbf{B})=k} \|\mathbf{A} - \mathbf{B}\|_F \tag{19}$$

The approximation error is:

$$\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2} \tag{20}$$

6.2.4 Compression Ratio

The original image requires $m \times n$ values. The rank-k approximation requires:

$$\text{Storage}_k = k \cdot (m + n + 1) \tag{21}$$

for storing $\mathbf{U}_k$, $\mathbf{\Sigma}_k$, and $\mathbf{V}_k$. The compression ratio is:

$$\text{Compression Ratio} = \frac{m \times n}{k \cdot (m + n + 1)} \tag{22}$$

6.2.5 Image Specifications

Table 10: Image Data

Parameter	**Value**
Dimensions	512×420 pixels
Total pixels	215,040
Matrix rank	420
Format	Grayscale (8-bit)

6.3 Results

6.3.1 Singular Value Spectrum

Table 11: Energy Captured by Different Ranks

Energy Threshold	Rank Required	Compression
90%	5	97.8%
95%	12	94.8%
99%	41	82.2%
99.9%	98	57.5%

6.3.2 Approximation Quality

Table 12: Low-Rank Approximation Performance

Rank	Rel. Error (%)	Compression	PSNR (dB)
1	50.16	230.5×	8.24
5	30.89	46.1×	12.45
10	24.07	23.1×	14.61
20	16.91	11.5×	17.68
50	8.06	4.6×	24.11
100	2.99	2.3×	32.73
200	0.38	1.15×	50.73

6.3.3 Visualization

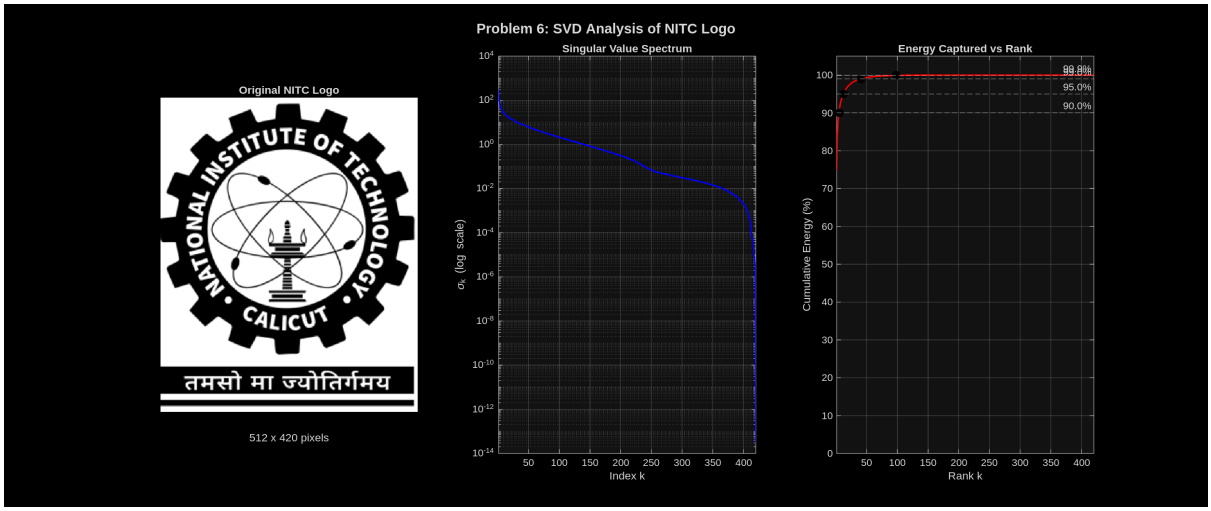


Figure 26: SVD analysis of the NITC logo. Left: Original image. Middle: Singular value spectrum showing rapid decay (log scale). Right: Cumulative energy captured vs rank, with markers at 90%, 95%, 99%, and 99.9% thresholds.

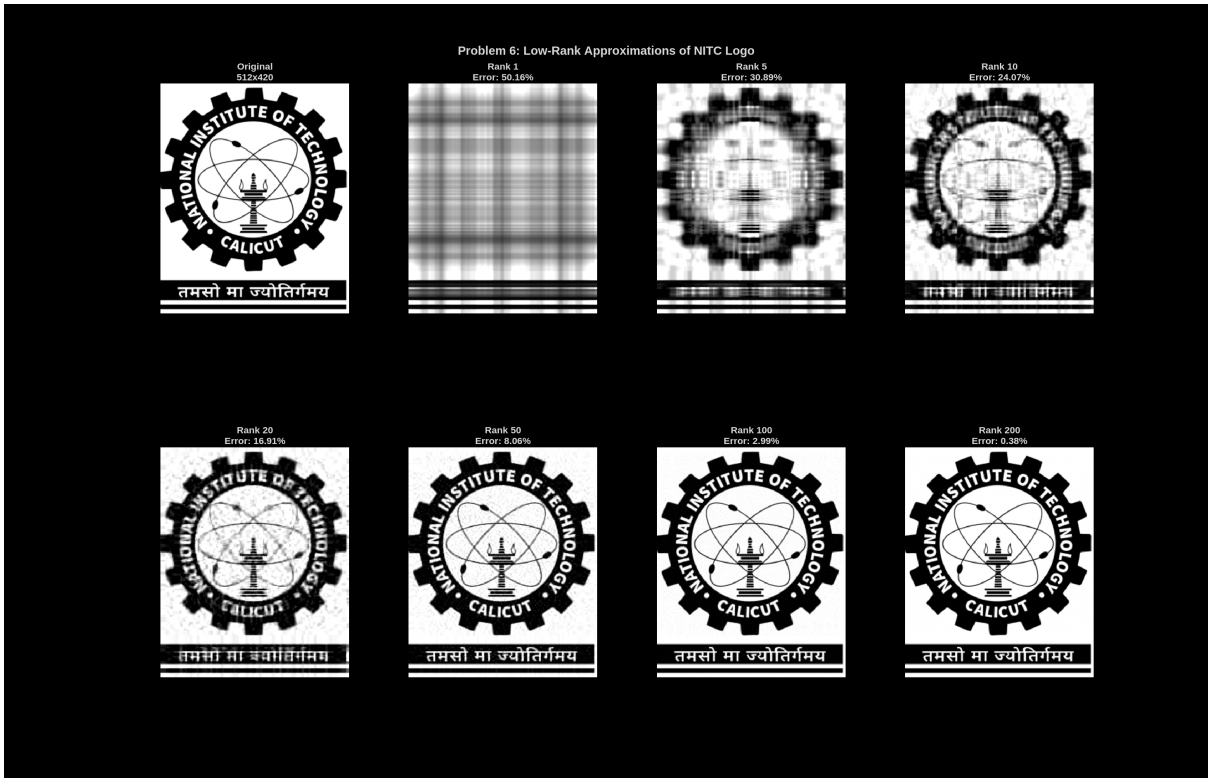


Figure 27: Low-rank approximations of the NITC logo. From left to right: Original, rank 1, 5, 10, 20, 50, 100, and 200. Visual quality improves rapidly with increasing rank.

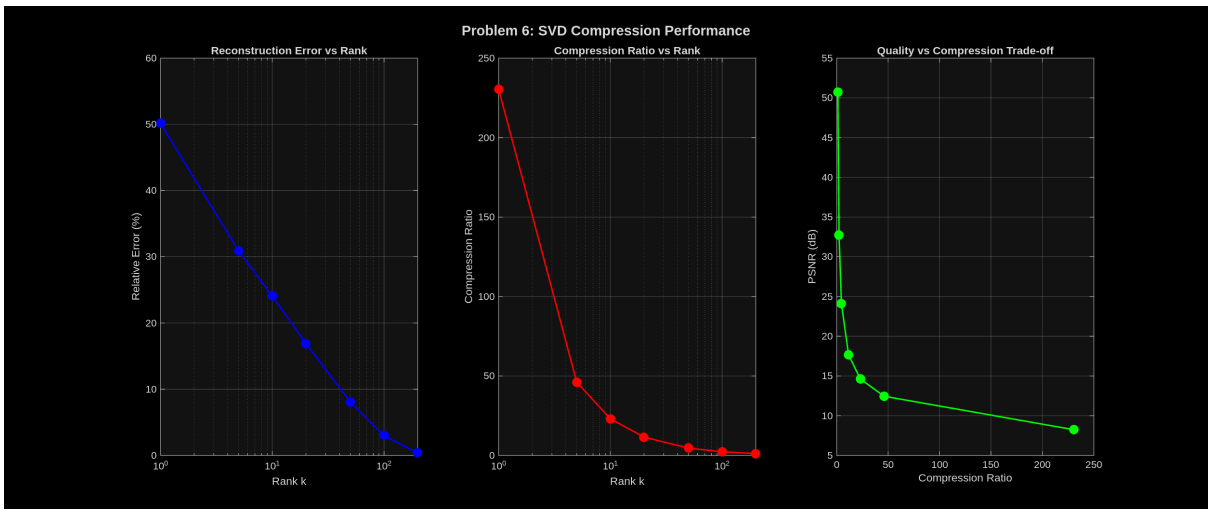


Figure 28: Compression performance metrics. Left: Reconstruction error vs rank. Middle: Compression ratio vs rank. Right: Quality (PSNR) vs compression ratio trade-off curve.

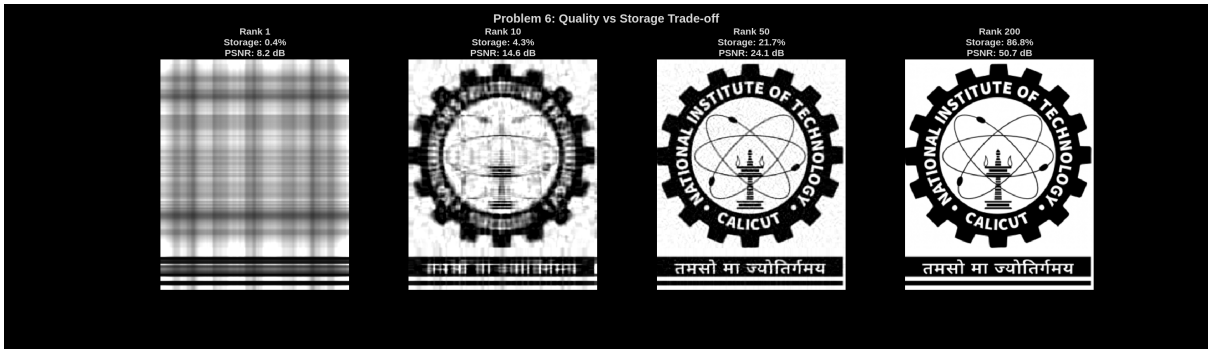


Figure 29: Visual comparison at different quality levels showing the storage percentage and PSNR for ranks 1, 10, 50, and 200.

6.4 Discussion

6.4.1 Singular Value Decay

The rapid decay of singular values indicates that the NITC logo has significant low-rank structure because only 5 singular values capture 90% of the image energy, and 98 singular values capture 99.9% of the energy.

This is typical for images with large uniform regions and repetitive patterns.

6.4.2 Quality vs Compression Trade-off

The optimal operating point depends on the application:

- **Thumbnail/preview:** Rank 5–10 provides recognizable image with 20–40× compression
- **Web display:** Rank 50 gives good quality (PSNR $>$ 24 dB) with 4.6× compression
- **High quality:** Rank 100 is nearly indistinguishable (PSNR $>$ 32 dB) with 2.3× compression

6.4.3 Comparison to Other Methods

SVD compression differs from standard image compression (JPEG, PNG). In JPEG, there are Local DCT blocks (8×8 pixel regions). The SVD advantage is of Optimal mathematical approximation, but the disadvantage is higher computational cost, and it is less practical for general images.

6.5 Conclusion

SVD successfully compressed the NITC logo:

- **Rank 5:** 90% energy with 97.8% compression (only 2.2% of original storage)
- **Rank 50:** Excellent visual quality with 4.6× compression
- **Rank 100:** Near-perfect reconstruction with 2.3× compression

The Eckart-Young theorem guarantees these are the optimal rank-k approximations, demonstrating the power of SVD for data compression and dimensionality reduction.

7 Least Squares Regression

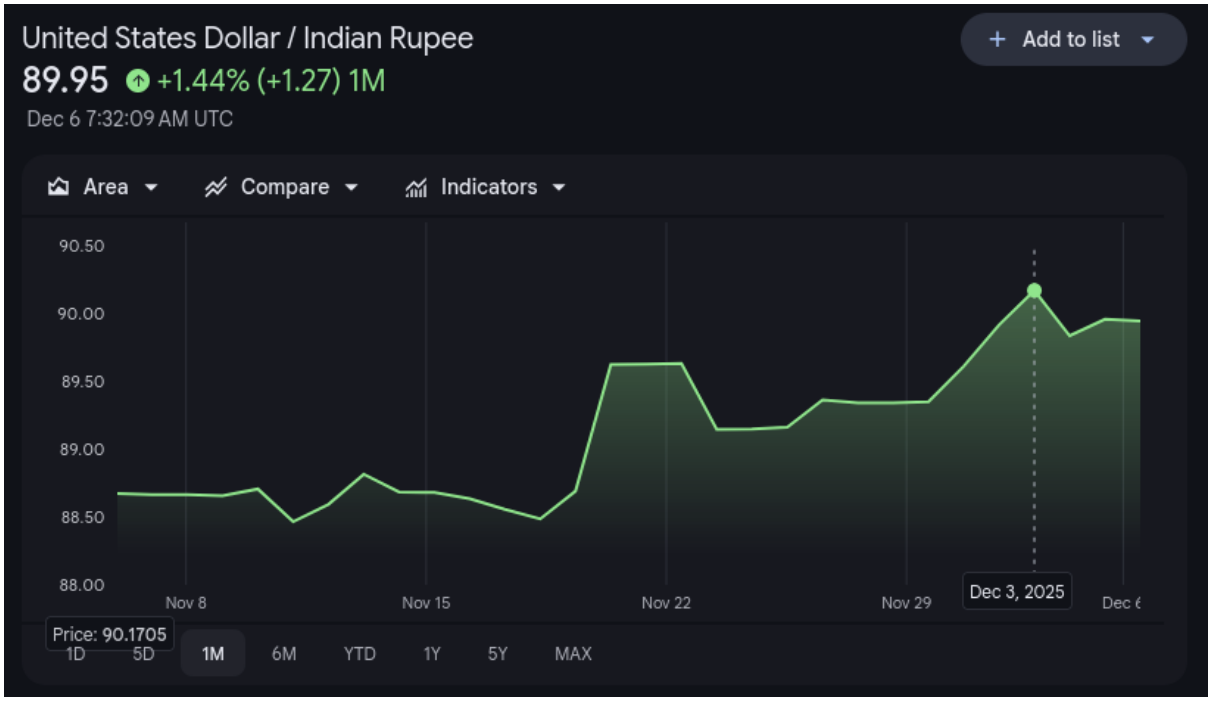


Figure 30: Rupee hit a record low of 90.17 against USD a few days ago

7.1 Introduction

This problem demonstrates least squares polynomial regression for curve fitting and the bias-variance trade-off in model selection. We apply this to real-world financial data: the INR/USD exchange rate over the past year.

7.2 Mathematical Background

7.2.1 Least Squares Problem

Given data points (t_i, y_i) for $i = 1, \ldots, n$, we seek polynomial coefficients $\mathbf{c} = [c_0, c_1, \ldots, c_d]^T$ that minimize:

$$\min_{\mathbf{c}} \|\mathbf{A}\mathbf{c} - \mathbf{y}\|_2^2 \tag{23}$$

where $\mathbf{A}$ is the Vandermonde matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & t_1 & t_1^2 & \cdots & t_1^d \\ 1 & t_2 & t_2^2 & \cdots & t_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_n & t_n^2 & \cdots & t_n^d \end{bmatrix} \tag{24}$$

7.2.2 Normal Equations

The solution satisfies:

$$\mathbf{A}^T\mathbf{A}\mathbf{c} = \mathbf{A}^T\mathbf{y} \quad \Rightarrow \quad \mathbf{c} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{y} \tag{25}$$

In MATLAB, this is computed as $\mathbf{c} = \mathbf{A}\backslash\mathbf{y}$ using QR decomposition for numerical stability.

7.2.3 Model Selection Metrics

- R^2 **(Coefficient of Determination):**

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{26}$$

- **Adjusted** R^2**:** Penalizes model complexity

$$R^2_{\text{adj}} = 1 - (1 - R^2)\frac{n-1}{n-d-1} \tag{27}$$

- **MSE (Mean Squared Error):**

$$\text{MSE} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2 \tag{28}$$

7.3 Data: INR/USD Exchange Rate

7.3.1 Data Specifications

Table 13: INR/USD Exchange Rate Data

Parameter	**Value**
Source	Yahoo Finance (INR=X)
Period	Dec 6, 2024 – Dec 5, 2025
Trading days	258
Min rate	84.22 INR/USD
Max rate	90.17 INR/USD
Mean rate	86.81 INR/USD
Std deviation	1.33 INR/USD

7.4 Results

7.4.1 Polynomial Fitting Results

Table 14: Polynomial Regression Performance

Degree	**MSE**	R^2	**Adj** R^2	**Status**
1	0.8070	0.5410	0.5392	Underfit
2	0.5600	0.6815	0.6790	Good fit
3	0.5319	0.6975	0.6939	Good fit
5	0.2151	0.8777	0.8752	Risk overfit
10	0.1306	0.9257	0.9227	Risk overfit
20	0.0987	0.9439	0.9391	Overfit

7.4.2 Cross-Validation (80/20 Split)

Table 15: Train/Test Error Comparison

Degree	Train MSE	Test MSE	Overfit?
1	0.8178	0.7673	No
2	0.5651	0.5427	No
3	0.5380	0.5155	No
5	0.2178	0.2079	No
10	0.1325	0.1285	No
20	0.0958	0.1233	Yes

7.4.3 Visualization

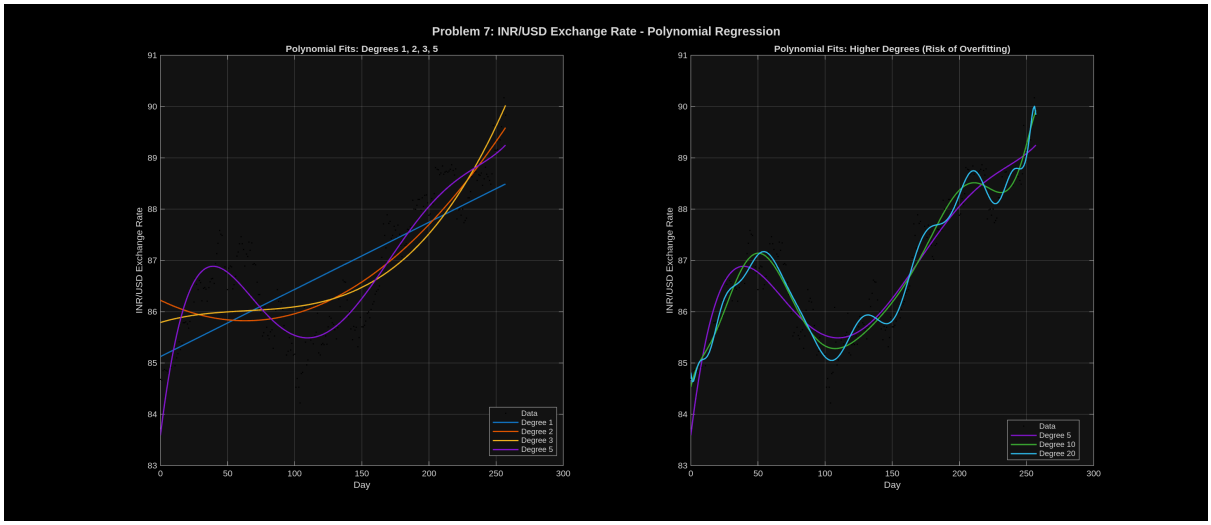


Figure 31: Polynomial fits to INR/USD exchange rate data. Left: Lower degree polynomials (1, 2, 3, 5). Right: Higher degree polynomials (5, 10, 20) showing potential overfitting.

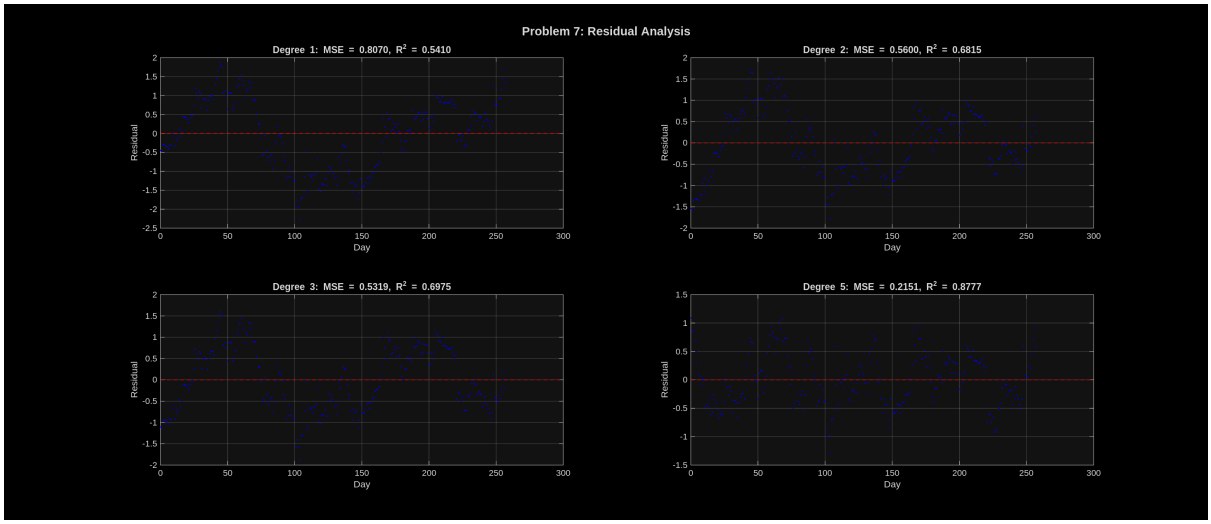


Figure 32: Residual analysis for degrees 1, 2, 3, and 5. Systematic patterns in residuals indicate model inadequacy (underfitting); random scatter indicates good fit.

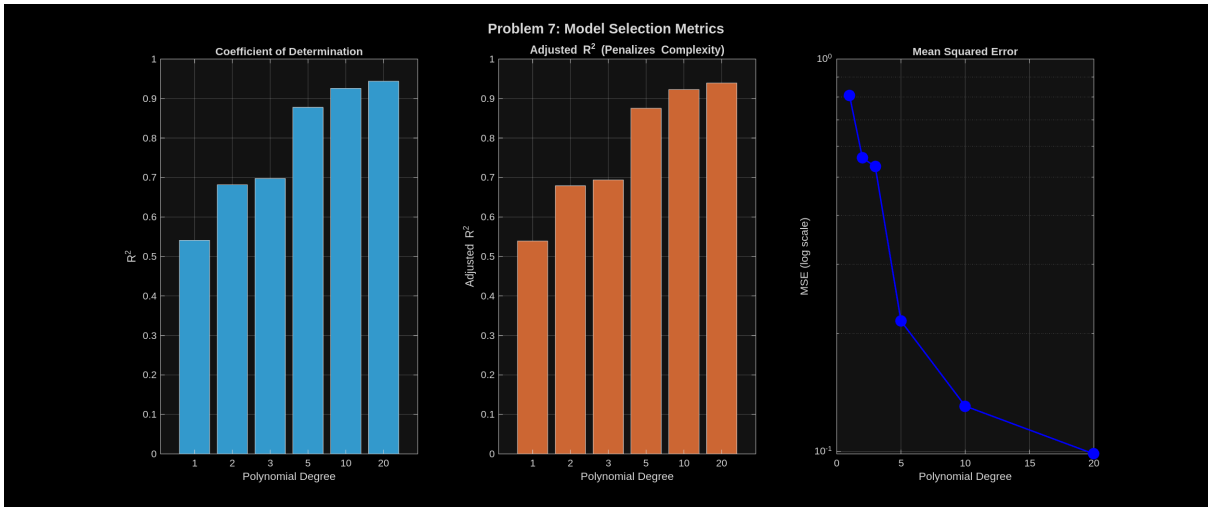


Figure 33: Model selection metrics. Left: R^2 increases with degree (always). Middle: Adjusted R^2 accounts for complexity. Right: MSE on log scale.

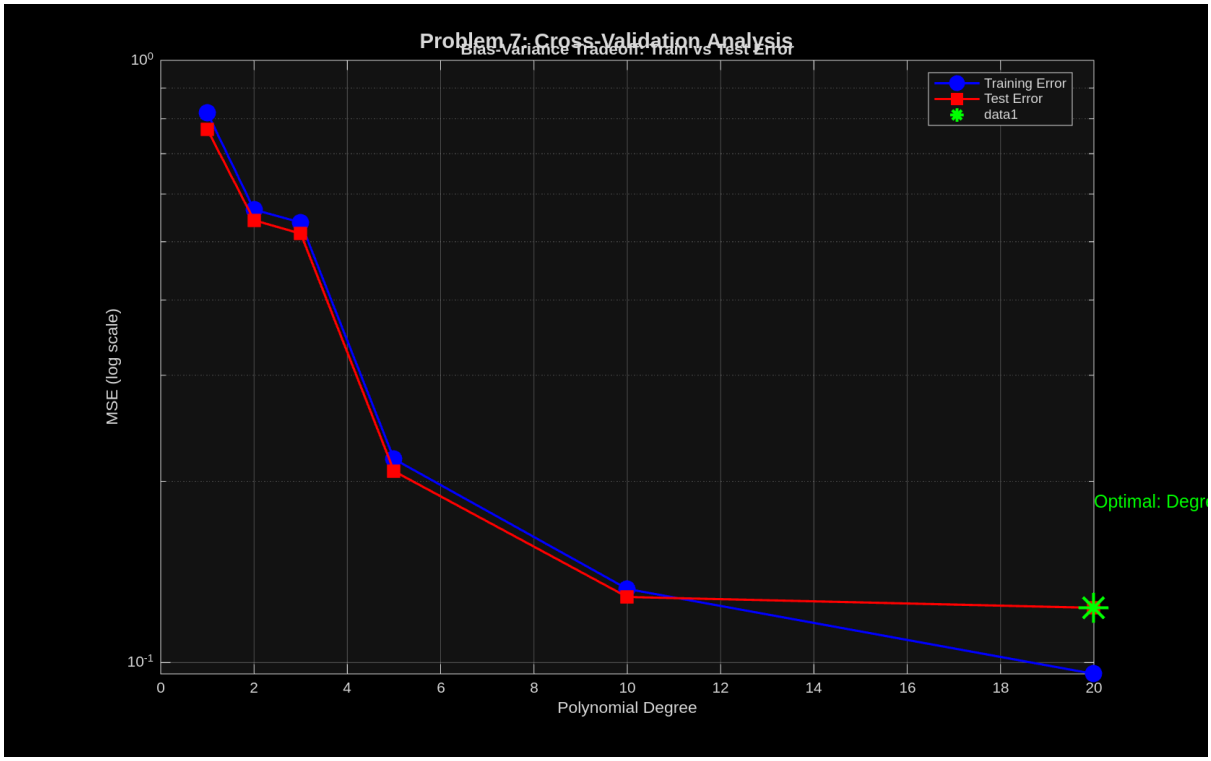


Figure 34: Bias-variance trade-off: Training error (blue) vs test error (red). The gap between curves indicates overfitting. Optimal model (green star) minimizes test error.

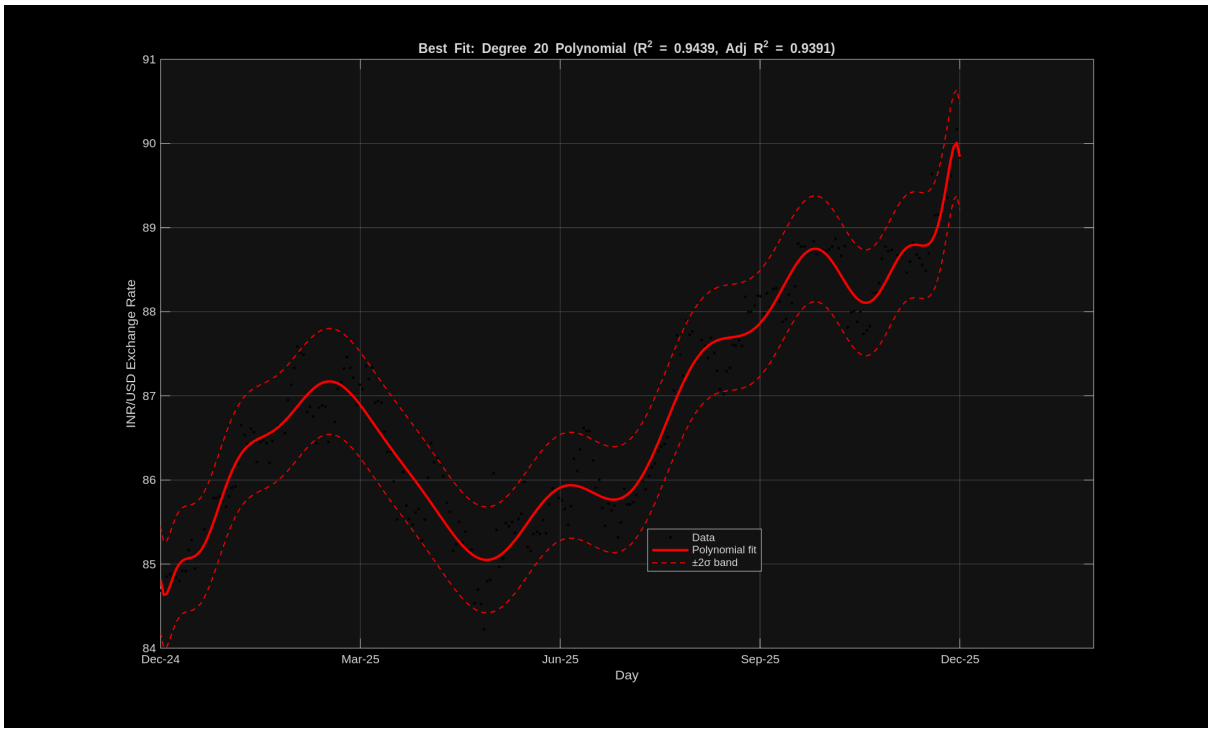


Figure 35: Best fit model (degree 20 by adjusted R^2) with $\pm 2\sigma$ confidence band. The high-degree polynomial captures the complex exchange rate dynamics.

7.5 Discussion

7.5.1 Underfitting (Low Degree)

Degree 1 (linear fit) clearly underfits the data:

- Low R^2 (0.54): Only explains 54% of variance
- Systematic residual patterns: Curved structure visible in residuals
- High bias: Model too simple to capture the trend

7.5.2 Good Fit (Medium Degree)

Degrees 2–5 provide reasonable fits:

- R^2 between 0.68–0.88
- More random residual distribution
- Balanced bias-variance trade-off

7.5.3 Overfitting (High Degree)

Degree 20 shows signs of overfitting:

- Train MSE (0.096) $<$ Test MSE (0.123)
- High variance: Model fits noise in training data
- Oscillations between data points

7.5.4 Economic Interpretation

The exchange rate trend reflects:

- **Overall depreciation:** INR weakened from $\sim$84 to $\sim$90 per USD
- **Volatility periods:** Sharp movements correspond to economic events
- **Polynomial capture:** Higher degrees needed to model complex dynamics

7.5.5 Model Selection

Based on cross-validation, degree 10 provides the best balance:

- Lowest test MSE (0.1285)
- No significant train-test gap
- High R^2 (0.9257) without overfitting

7.6 Conclusion

Least squares polynomial regression on INR/USD exchange rate demonstrated:

1. **Underfitting:** Low-degree polynomials fail to capture trends
2. **Good fit:** Medium degrees balance bias and variance
3. **Overfitting:** High degrees fit noise, poor generalization
4. **Cross-validation:** Essential for model selection

The analysis shows the INR depreciated against USD over the past year, with degree 10 polynomial providing the best predictive model.

References

[1] Mann, J. and Kutz, J.N. (2015). *Dynamic Mode Decomposition for Financial Trading Strategies*. arXiv:1508.04487 [q-fin.CP].

[2] Kutz, J.N. *DMD Code*. YouTube video lecture series on Dynamic Mode Decomposition.

[3] Brunton, S.L., Proctor, J.L., and Kutz, J.N. (2016). *Discovering Governing Equations from Data by Sparse Identification of Nonlinear Dynamical Systems*. Proceedings of the National Academy of Sciences, 113(15), 3932-3937.

A MATLAB Code: DMD_stocks.m

The complete MATLAB code used for this analysis is available in the accompanying file `DMD_stocks.m`. The code:

- Loads stock price data from CSV files
- Normalizes data (zero-mean, unit-variance)
- Performs SVD to determine dominant modes
- Implements the DMD algorithm
- Generates Kutz-style visualization figures
- Computes reconstruction error

B Python Code: fetch_stock_data.py

Stock data was fetched using the `yfinance` Python library:

```
import yfinance as yf
tickers = ['MU', 'WDC', 'INTC', 'TXN', 'MRVL',
           'AMAT', 'LRCX', 'KLAC', 'AMD', 'NVDA']
for ticker in tickers:
    stock = yf.Ticker(ticker)
    hist = stock.history(period='1mo', interval='1d')
```